Scripting FPGA Bitfiles from CSV Files

Daniel Eaton

EMERSON

Daniel Eaton

Chief Field Applications Engineer

CLA, CLED, 20 years “doing” w/ NI

Daniel.eaton@emerson.com

We are here to help!

- Pull us in to design conversations
- Let us help de-risk projects
- Tell us what you want to learn about next

Content is on GITHUB

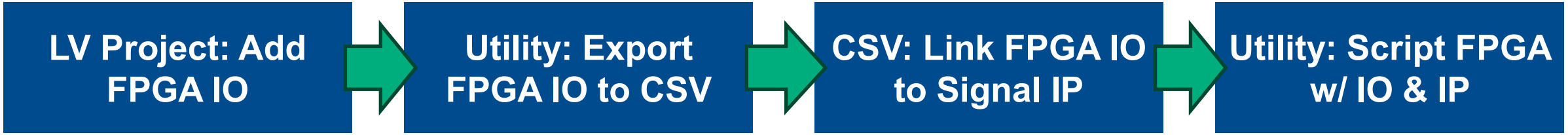
https://github.com/NISystemsEngineering/ni-LunchAndLearn-ScriptLvFpgaFromCsvExamples

Goals for Today

The Result	DEMO!
The Basics	VI Server
	VI Scripting
	Common Actions
	LabVIEW Project Modification
The Code	Review

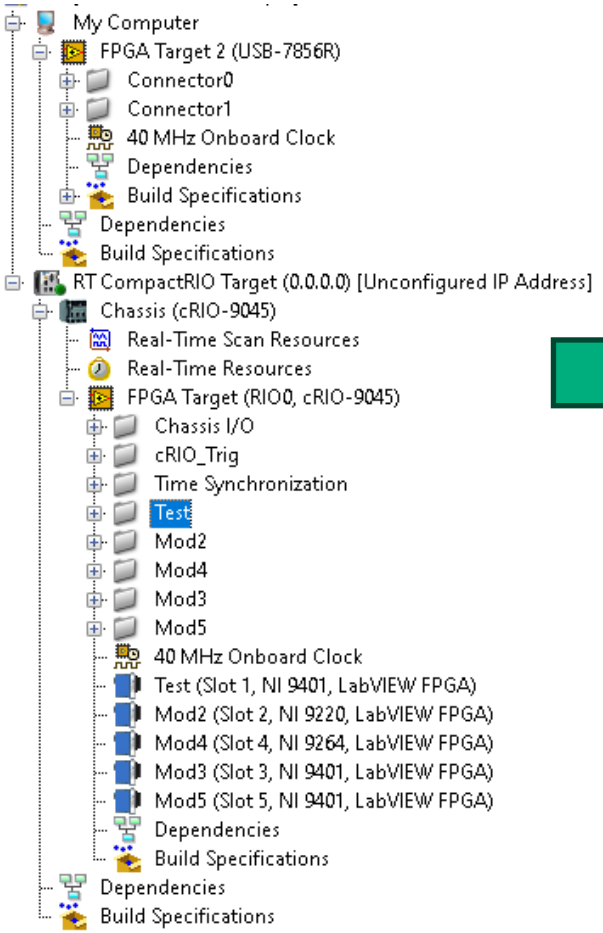
But First…The Demo

The Workflow

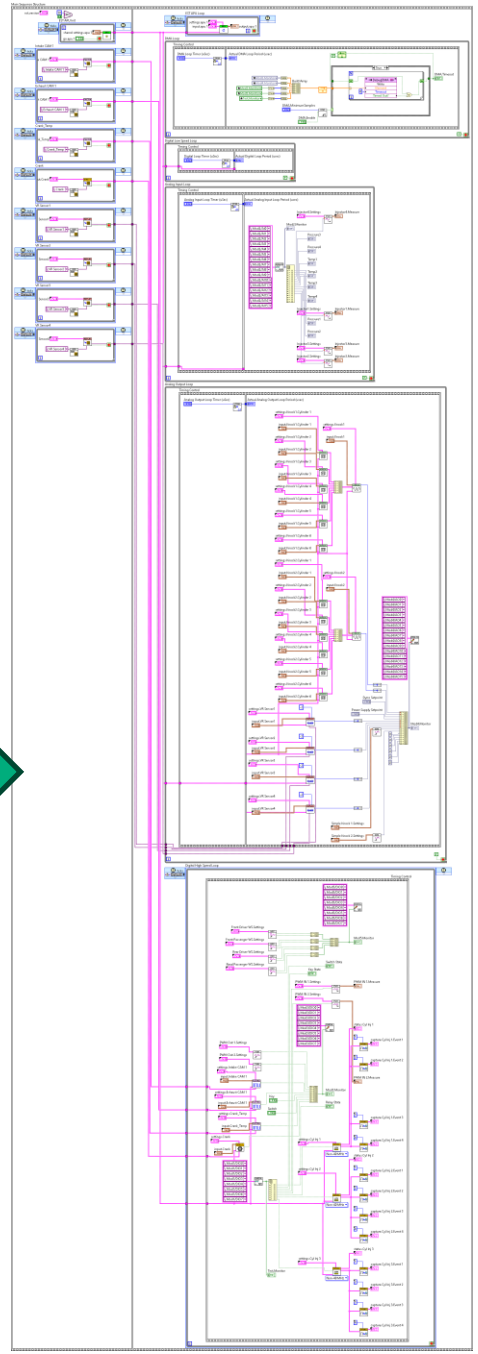


But First…The Demo

The Workflow



	A	B	C	D	E	F	G	H
1		Mod.Number	Group.Name	IO.Resource	IO.Name	Signal.Name	Signal.Type	Signal.Parameters
2	FPGA Target	9220	Mod2	AI0	Mod2/AI0	Pressure1	Analog Indicator	
3	FPGA Target	9220	Mod2	AI1	Mod2/AI1	Pressure2	Analog Indicator	
4	FPGA Target	9220	Mod2	AI2	Mod2/AI2	Injector3	Analog PWM Input	
5	FPGA Target	9220	Mod2	AI3	Mod2/AI3	Injector4	Analog PWM Input	
6	FPGA Target	9220	Mod2	AI4	Mod2/AI4	Pressure3	Analog Indicator	
7	FPGA Target	9220	Mod2	AI5	Mod2/AI5	Pressure4	Analog Indicator	
8	FPGA Target	9220	Mod2	AI6	Mod2/AI6	Temp1	Analog Indicator	
9	FPGA Target	9220	Mod2	AI7	Mod2/AI7	Temp2	Analog Indicator	
10	FPGA Target	9220	Mod2	AI8	Mod2/AI8	Temp3	Analog Indicator	
11	FPGA Target	9220	Mod2	AI9	Mod2/AI9	Temp4	Analog Indicator	
12	FPGA Target	9220	Mod2	AI10	Mod2/AI10	Injector1	Analog PWM Input	
13	FPGA Target	9220	Mod2	AI11	Mod2/AI11	Injector2	Analog PWM Input	
14	FPGA Target	9220	Mod2	AI12	Mod2/AI12			
15	FPGA Target	9220	Mod2	AI13	Mod2/AI13			
16	FPGA Target	9220	Mod2	AI14	Mod2/AI14			
17	FPGA Target	9220	Mod2	AI15	Mod2/AI15			
18	FPGA Target	9401	Mod3	DIO0	Mod3/DIO0	PWM Out 1	Digital PWM Output	
19	FPGA Target	9401	Mod3	DIO1	Mod3/DIO1	PWM Out 2	Digital PWM Output	
20	FPGA Target	9401	Mod3	DIO2	Mod3/DIO2	Intake CAM 1	Digital Pattern	
21	FPGA Target	9401	Mod3	DIO3	Mod3/DIO3	Exhaust CAM 1	Digital Pattern	
22	FPGA Target	9401	Mod3	DIO4	Mod3/DIO4	Key	Digital Control	
23	FPGA Target	9401	Mod3	DIO5	Mod3/DIO5	Switch	Digital Control	
24	FPGA Target	9401	Mod3	DIO6	Mod3/DIO6	Crank_Temp	Digital Pattern	
25	FPGA Target	9401	Mod3	DIO7	Mod3/DIO7	Crank	Directional Sensor	
26	FPGA Target	9264	Mod4	AO0	Mod4/AO0	Knock1	Knock Sensor	Cylinders:6
27	FPGA Target	9264	Mod4	AO1	Mod4/AO1	Knock2	Knock Sensor	Cylinders:6
28	FPGA Target	9264	Mod4	AO2	Mod4/AO2	Dyno Setpoint	Analog Control	
29	FPGA Target	9264	Mod4	AO3	Mod4/AO3	Power Supply Setpoint	Analog Control	



But First…The Demo

The Workflow

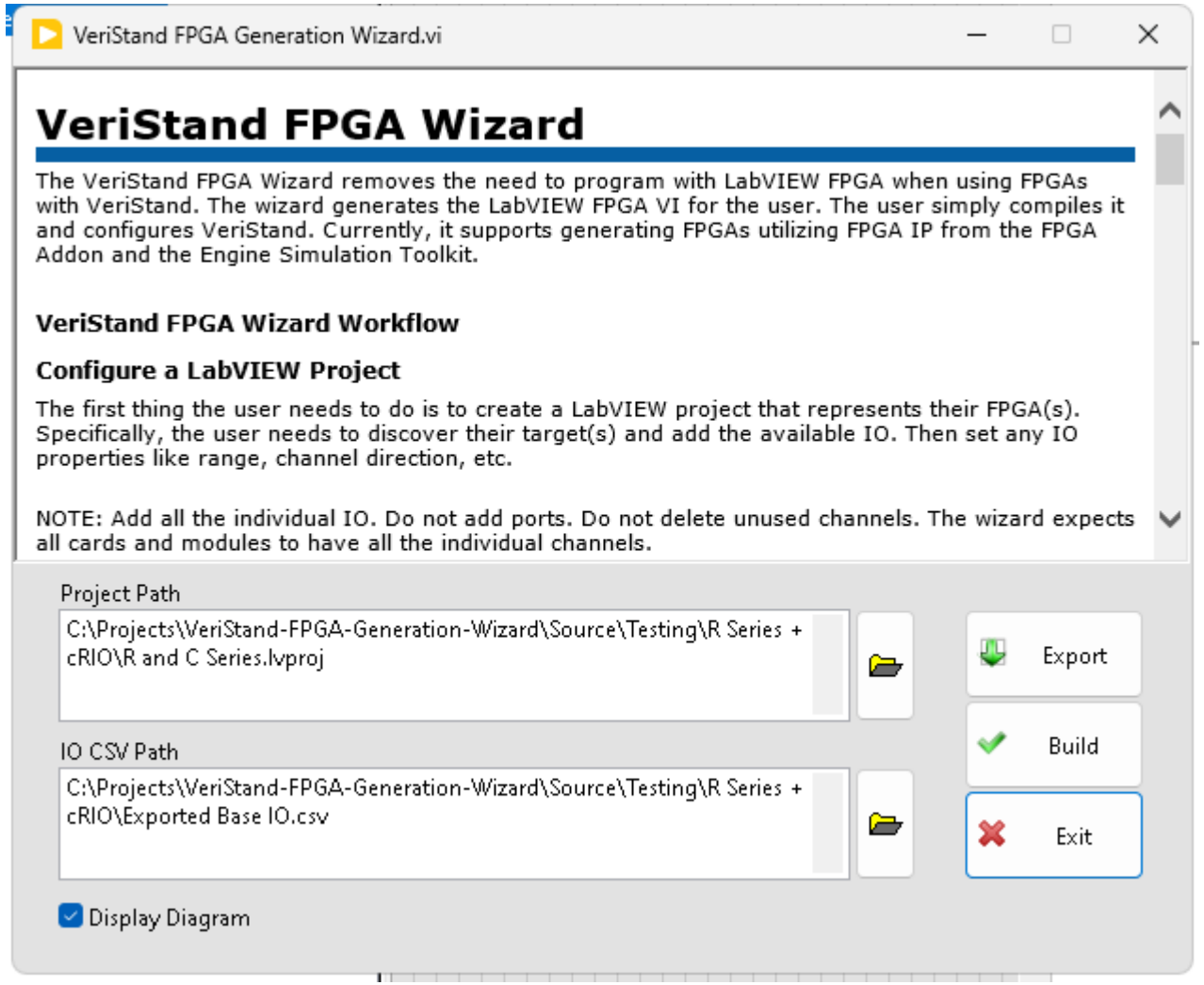
Open project: Testing\R series + cRIO\R and C Series.lvproj

- Show configured FPGA IO

Open project: VeriStand FPGA Scripting

Run VeriStand FPGA Generation Wizard.vi

- Select project: Testing\R Series + cRIO\R and C Series.lvproj
- Set CSV path: Testing\R Series + cRIO\Exported Base IO.csv
- Set Display Diagram to TRUE
- Hit Export



But First…The Demo

The Workflow

Open “Exported Base IO.csv”

- Show the base IO matches the LV Project
- Close “Exported Base IO.csv”

Open “Exported IO.csv”

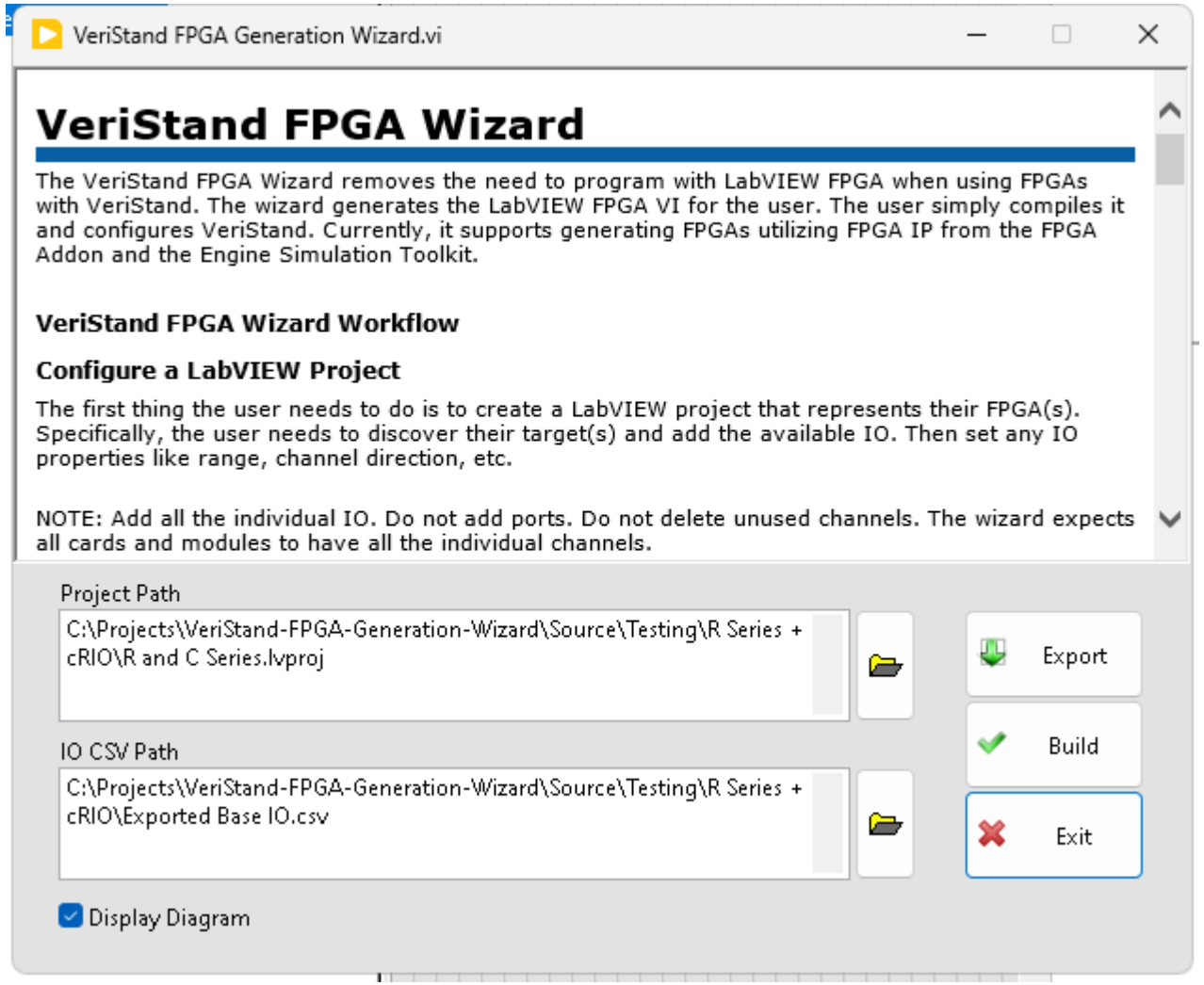
- Show IO have Signal IP mapped
- Show Signal IP in VeriStand FPGA Scripting.lvproj
- Close “Exported IO.csv”

Close Excel (CSV Files) or you will get Error 5

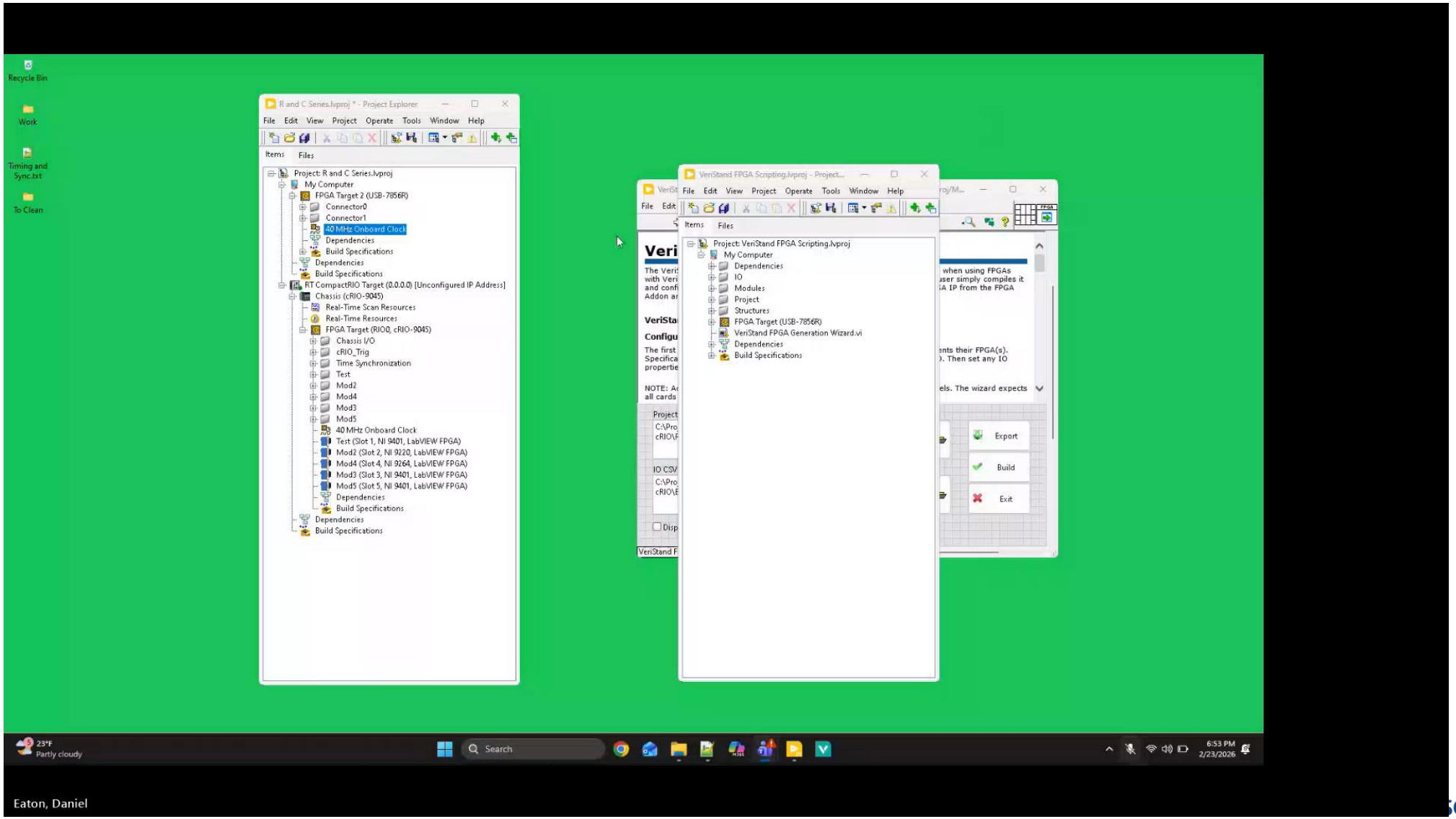
Set CSV path: Testing\R Series + cRIO\Exported IO.csv

Hit Build

Watch Program Generate



But First…The Demo



Eaton, Daniel

The Basics: VI Server

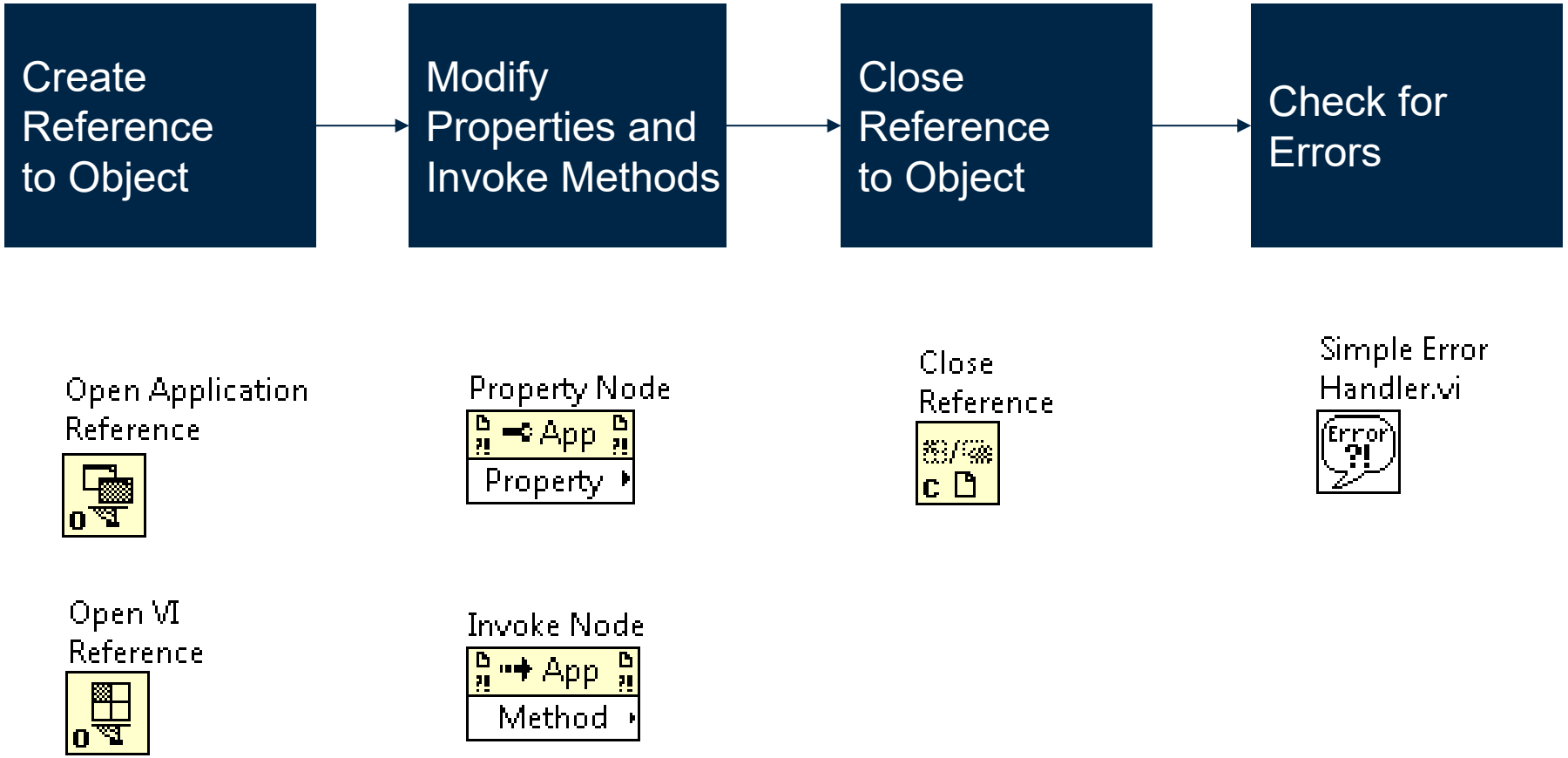
VI Server Terminology

VI Server – An object-oriented, platform-independent technology that provides programmatic access to LabVIEW and LabVIEW applications.

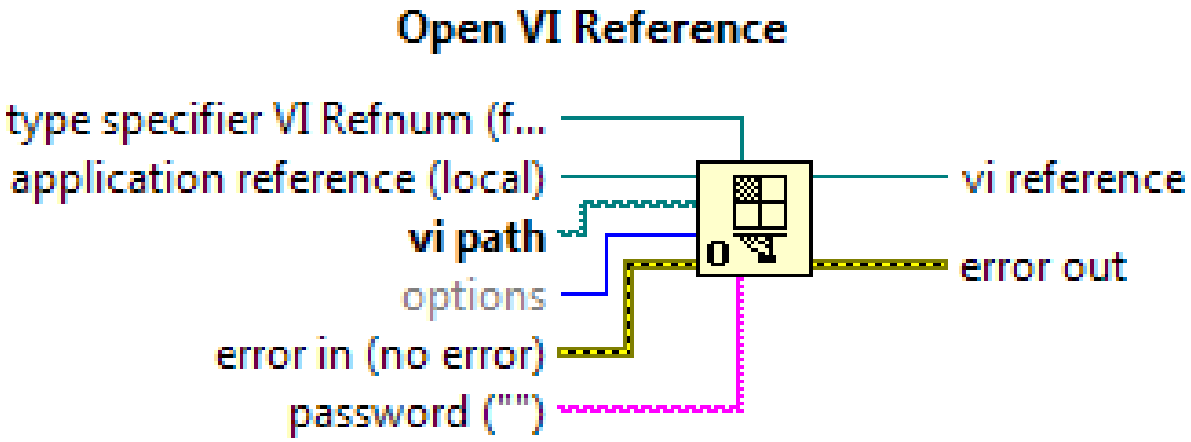
Property – A single-valued attribute of an object. Properties may be read/write, read only, or write only.

Methods – Functions that operate on an object.

VI Server Programming Model



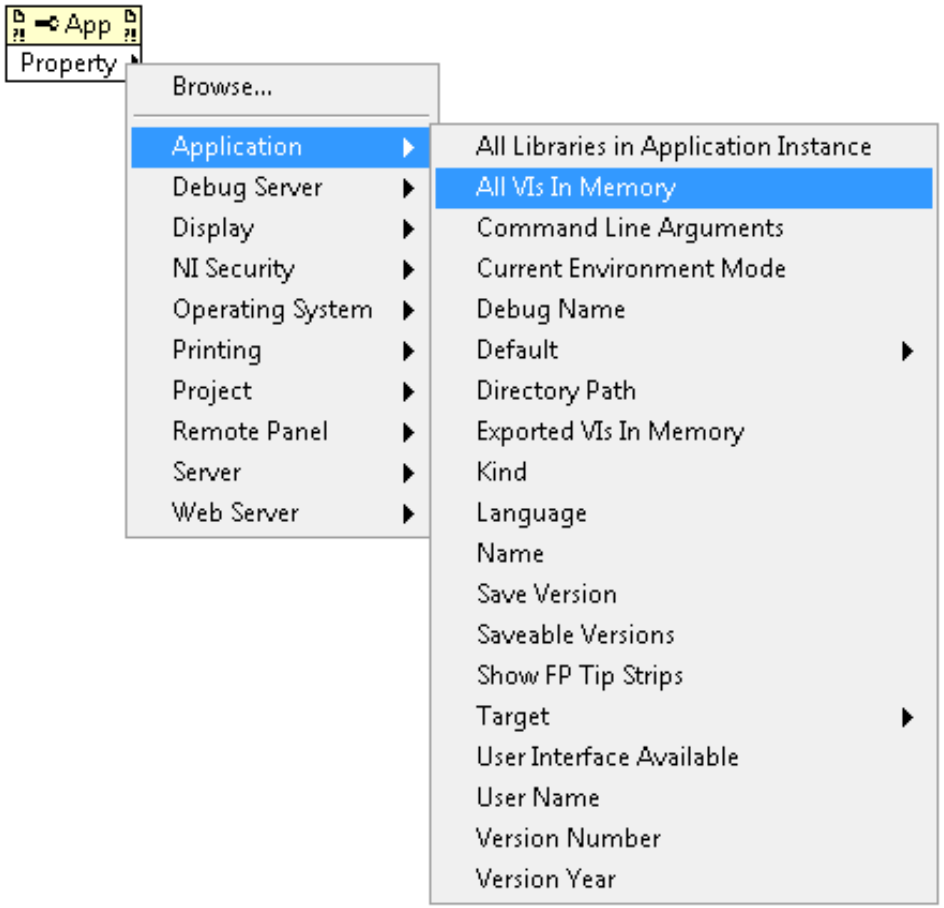
Open VI Reference



- Opening a VI reference creates a connection to a VI loaded in the LabVIEW environment
  - To reference a VI on a local computer, do not wire data to application reference.
  - To reference a VI on a remote computer, use Open Application Reference before Open VI Reference.

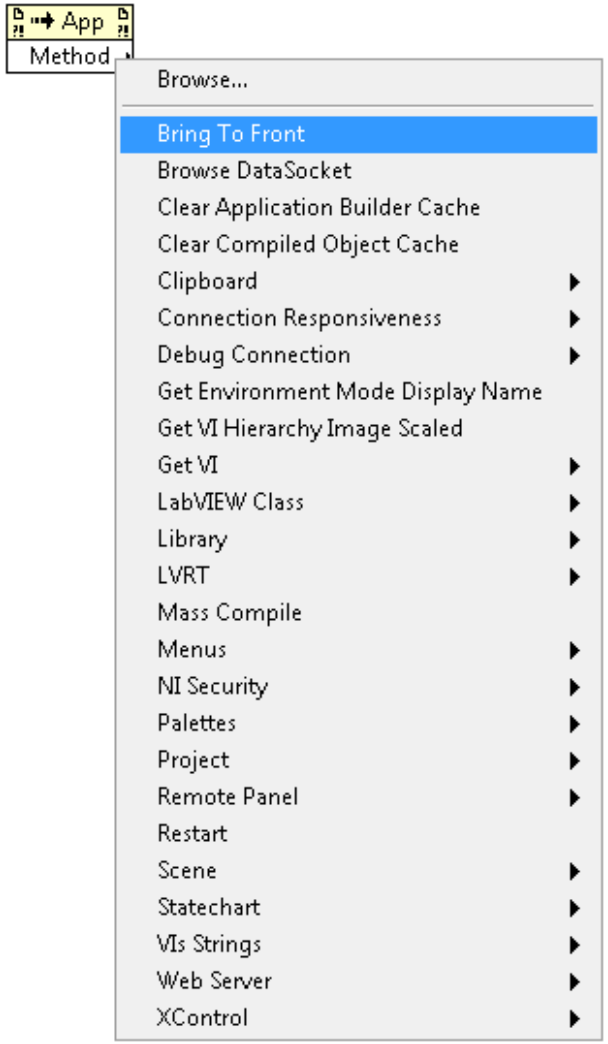
Property Nodes

- Use to get and set various properties of an application or a VI.
- You can read or write multiple properties using a single node.
- Properties are read/written in order from the top-down.



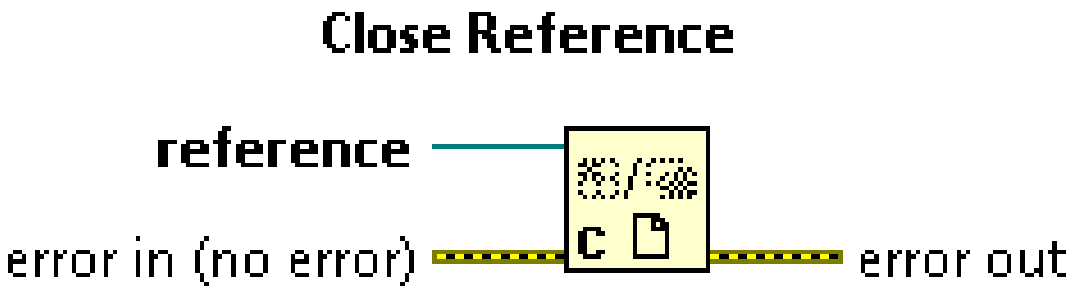
Invoke Nodes

- Perform actions or methods on an application or VI.
- A single Invoke Node executes a single method on an application or VI.



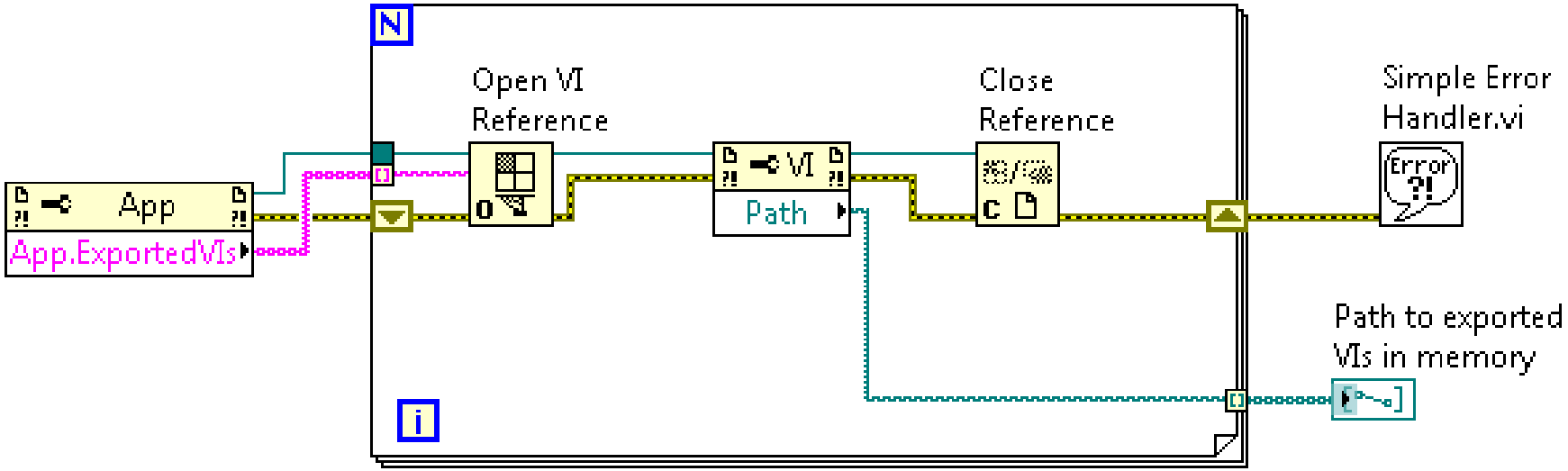
Close Reference Function

- Closing a reference closes the connection to the referenced resource
  - Used for both application references and VI references.
  - Closing a reference may unload a VI from memory.
- Does NOT prompt to save changes by default!



Example: Using Application and VI Class Properties

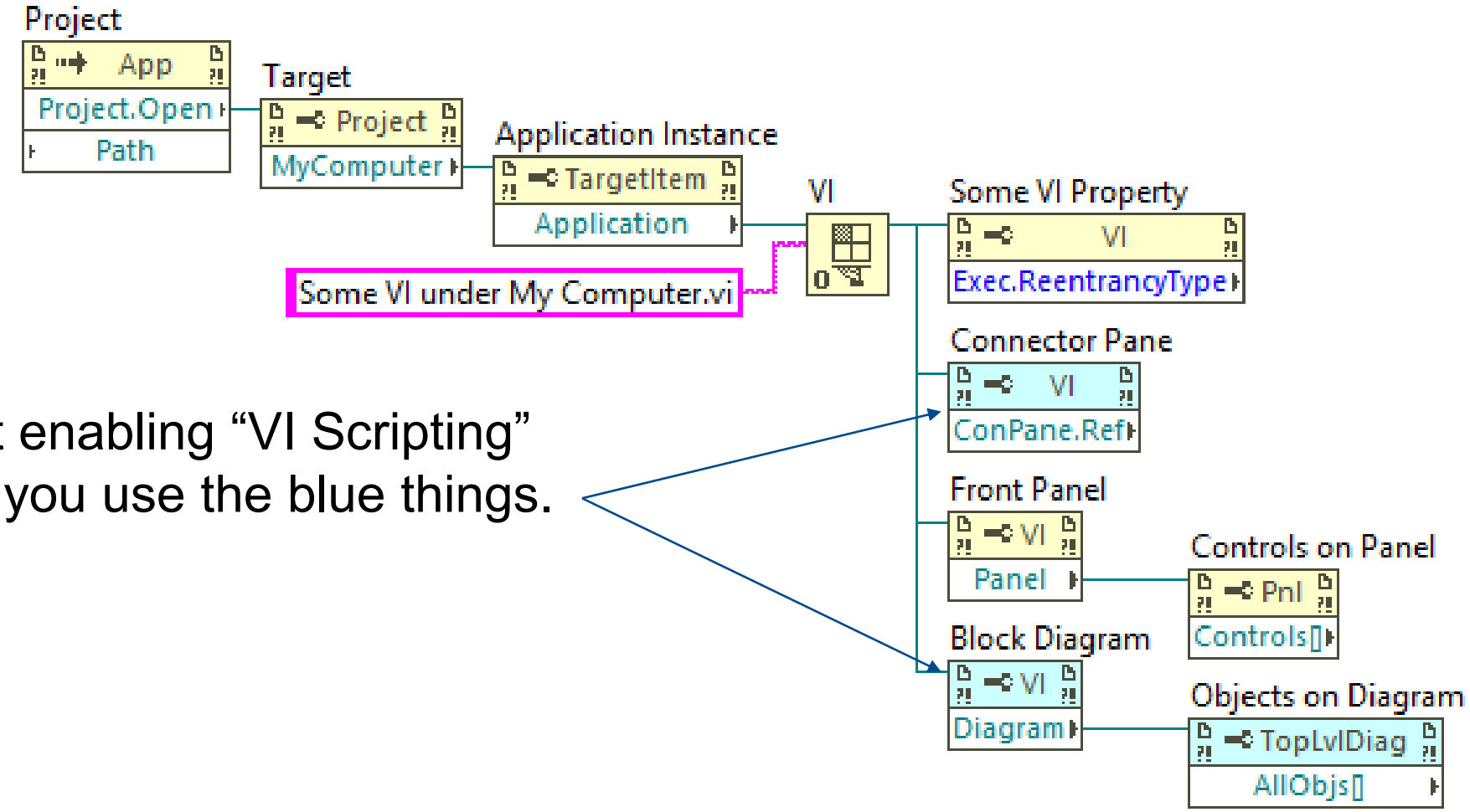
- Often you must combine elements from the application and VI classes.
- Application properties and methods can give you access to VIs in memory.
- VI properties and methods allow you to access or change VIs.



The Basics: VI Scripting

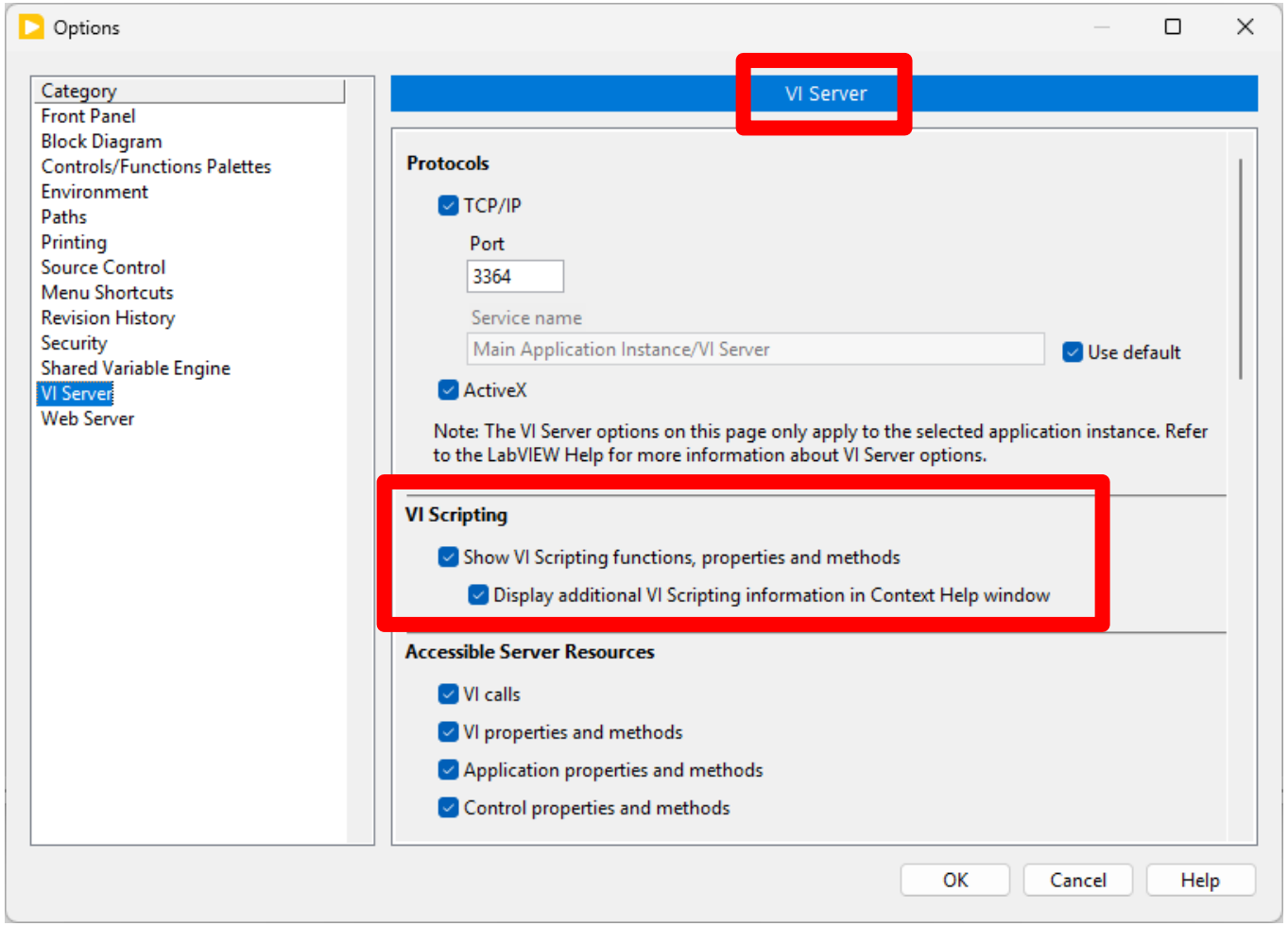
EMERSON

VI Scripting is VI Server. Just blue.

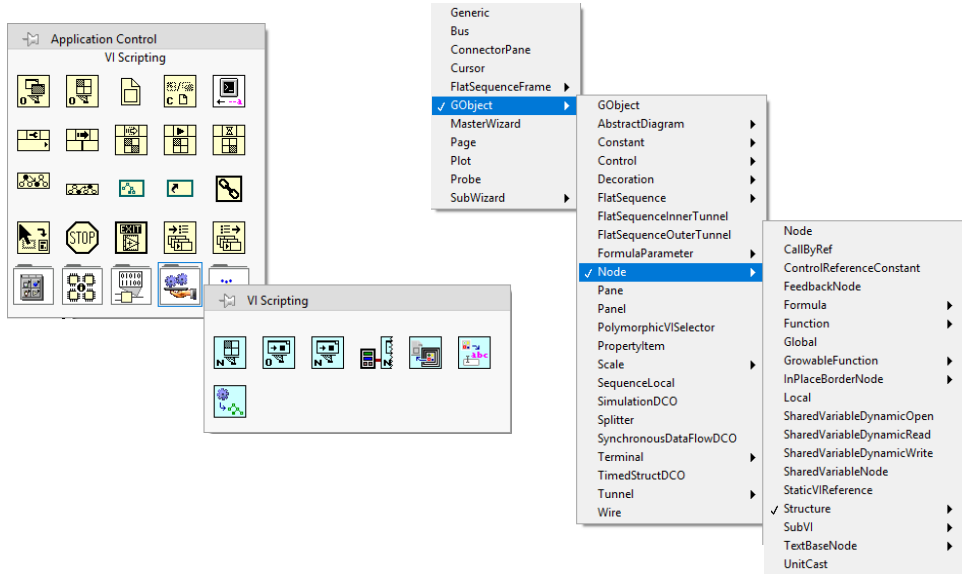


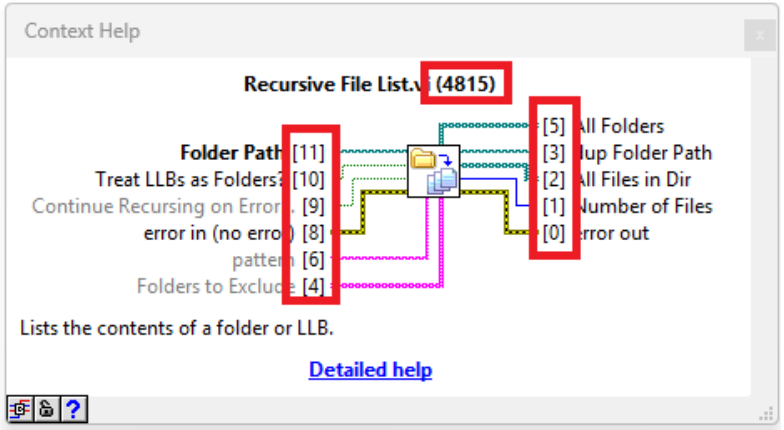
The **only** thing that enabling “VI Scripting” *actually* does is let you use the blue things.

How Do You Enable VI Scripting?



(by default, these options are *unchecked*)





The Property Node Color Code (also applies to Invoke Nodes)

Vanilla property node
Color = yellow
Status = boring

Scripting property node
Color = blue
Status = fun!

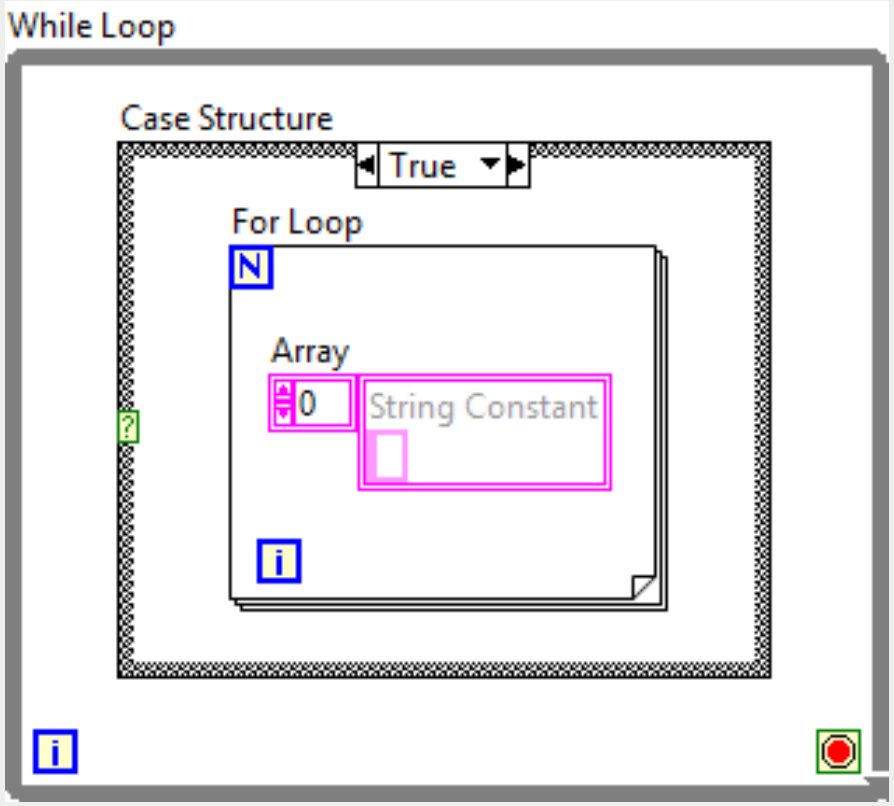
Private property node
Color = brown
Status = maybe fun?

Deprecated property node
Color = peach?
Status = tentatively ok

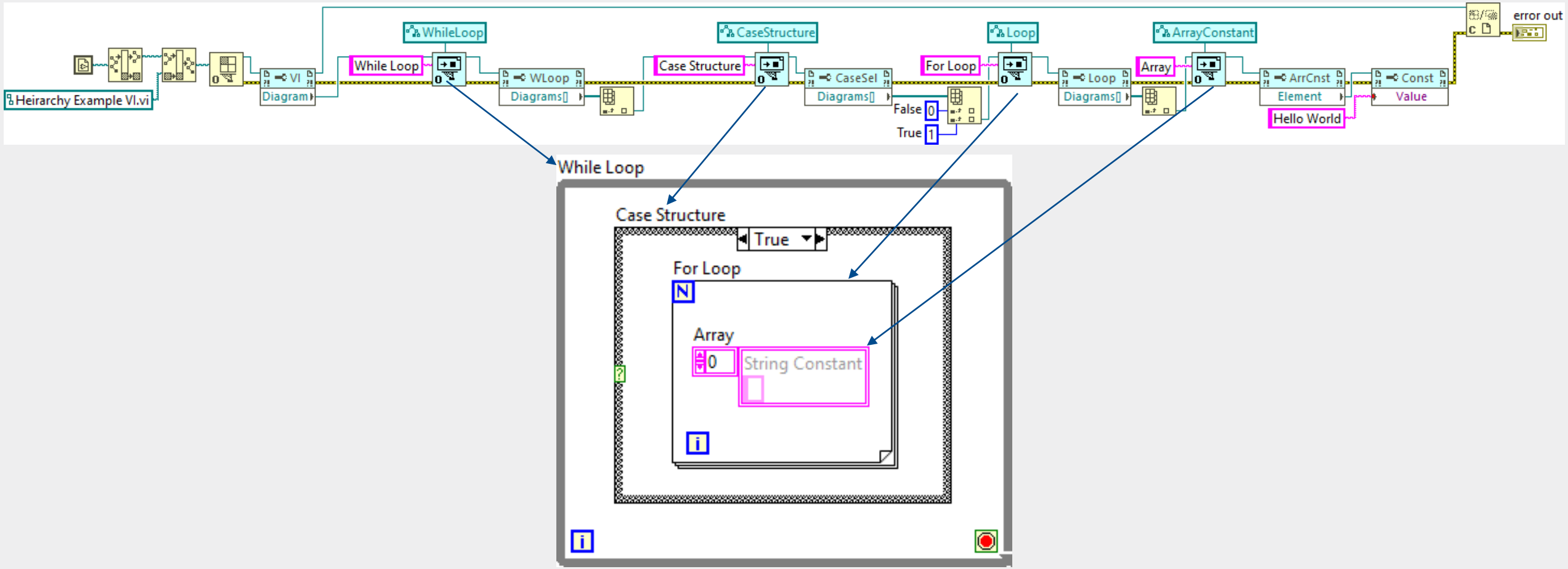
Owner Hierarchy on the block diagram

Top Level Diagram owns

- **While Loop** which owns
- **(Diagram)** which owns
- **Case Structure** which owns
- **(Diagram)** which owns
- **For Loop** which owns
- **(Diagram)** which owns
- **Array** which owns
- **String Constant**

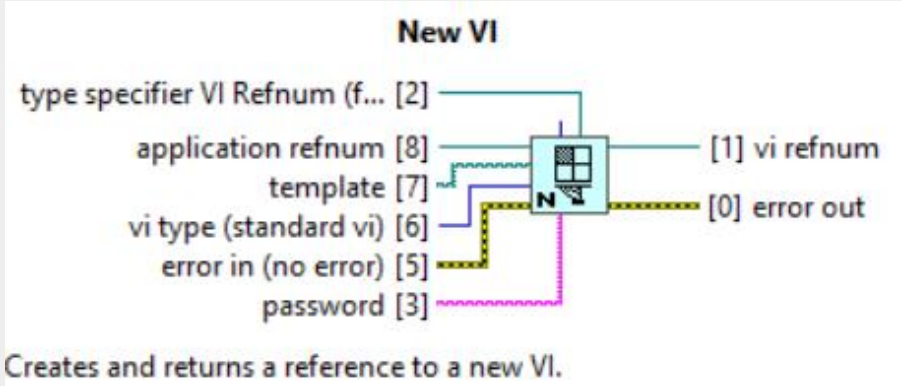


Owner Hierarchy on the block diagram

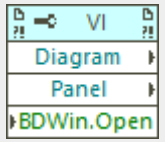


Core Functions

New VI



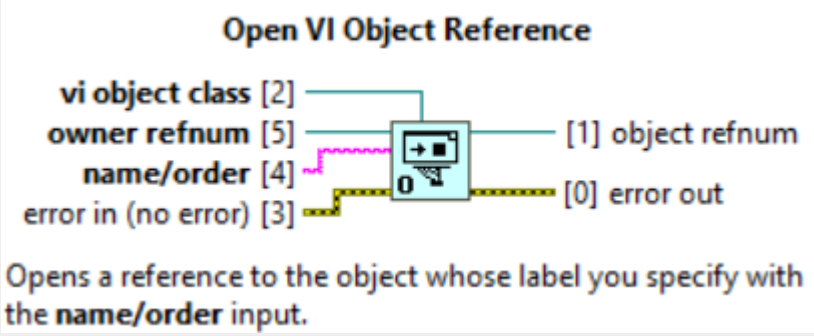
Getting Block Diagram & Front Panel



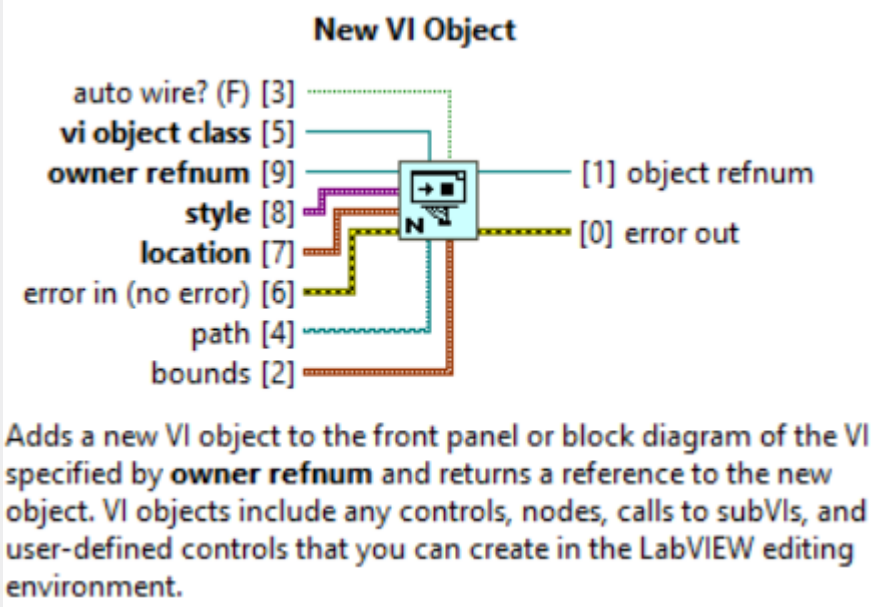
Core Functions

Object Interaction

Open VI Object

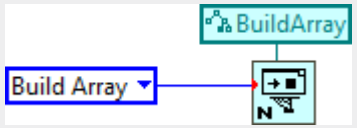


New VI Object

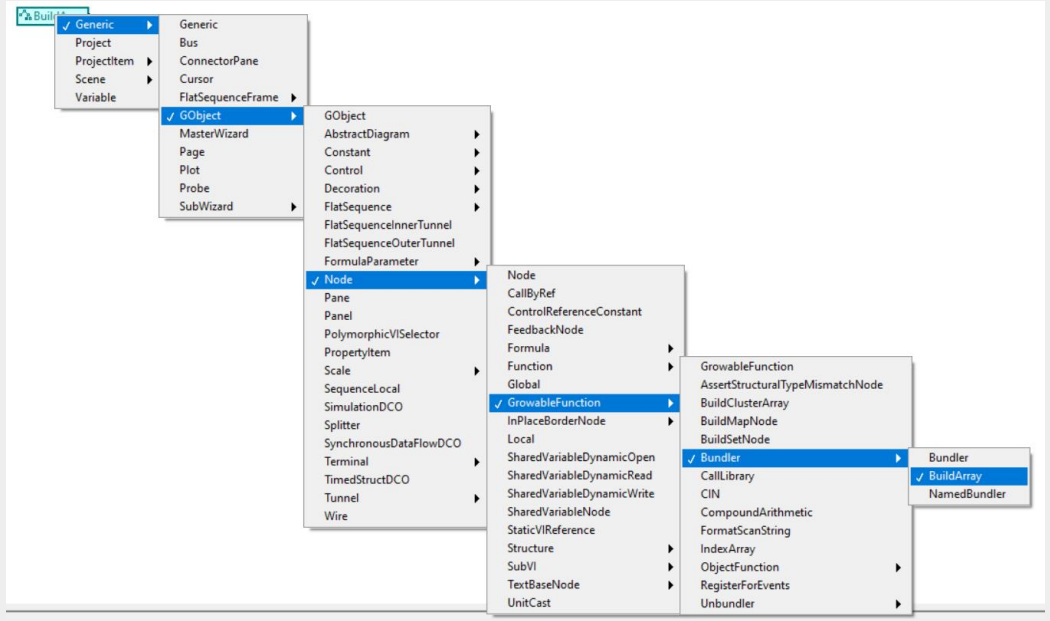


Core Functions…The Hardest Part…

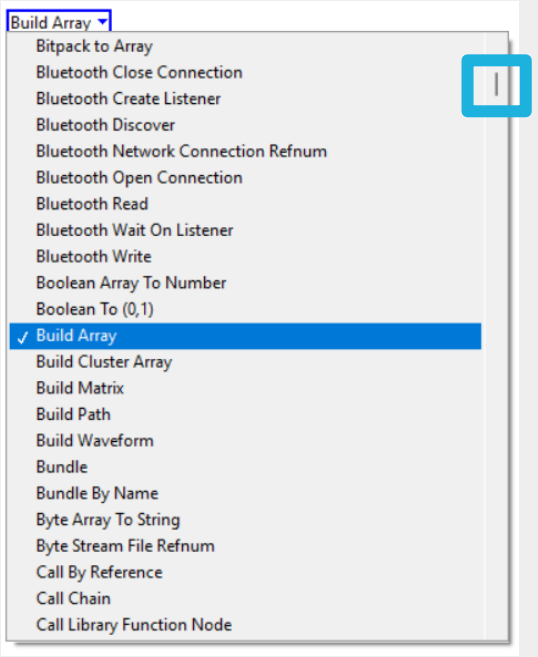
Goal:



VI Object Class – So many navigations…



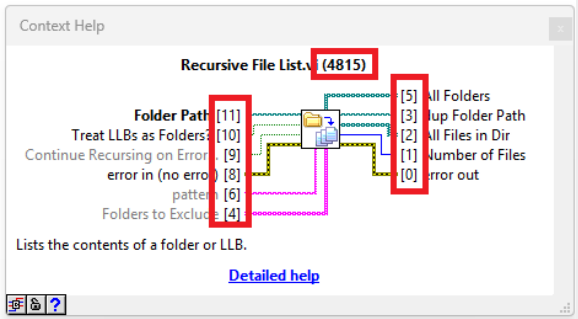
Style – So many names…



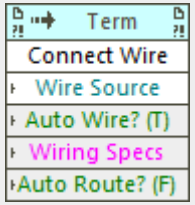
Wiring Functions

Connector Panes

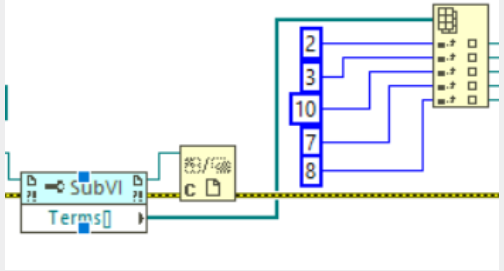
Use Context Help to understand terminal array index



Connecting wires



Use Terms[] to get terminals



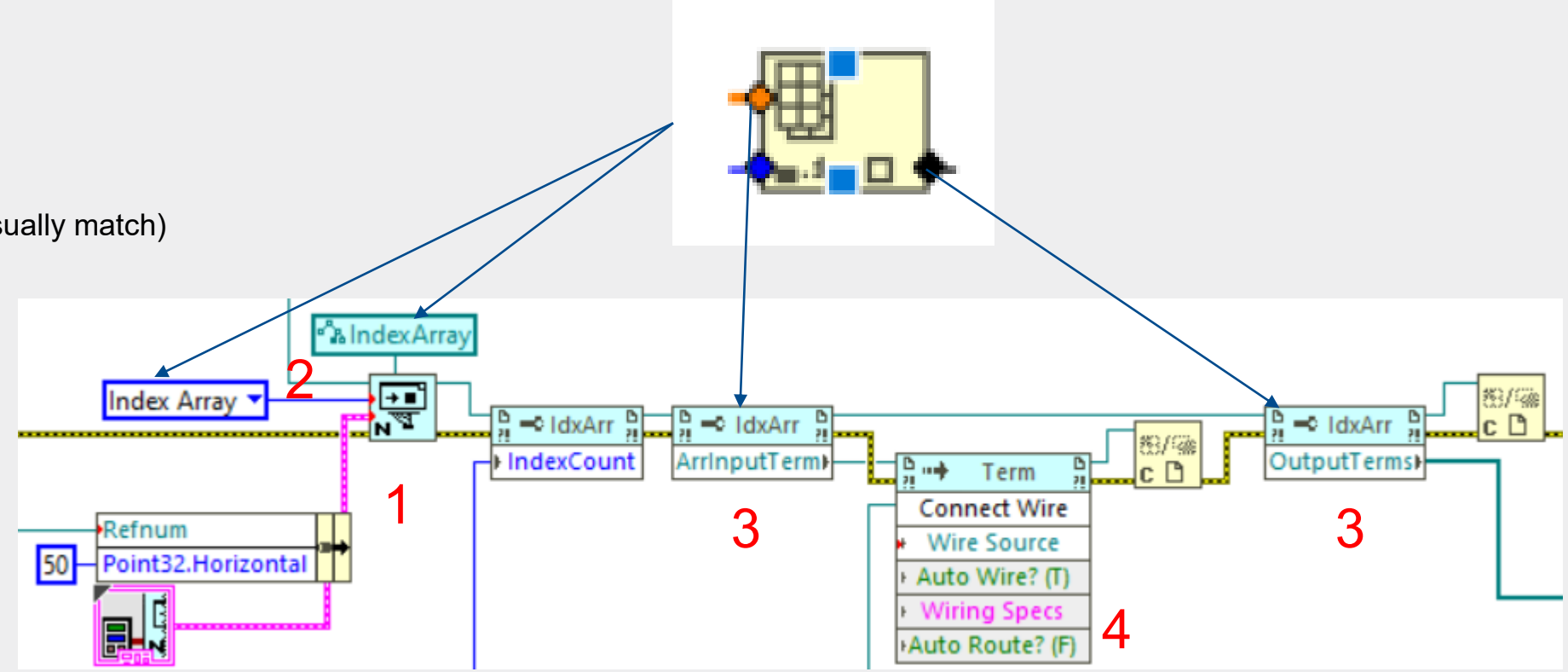
Creating Constant, Controls, and Indicators off Terminals

The Basics: Common Actions

Adding Items

Palette Items

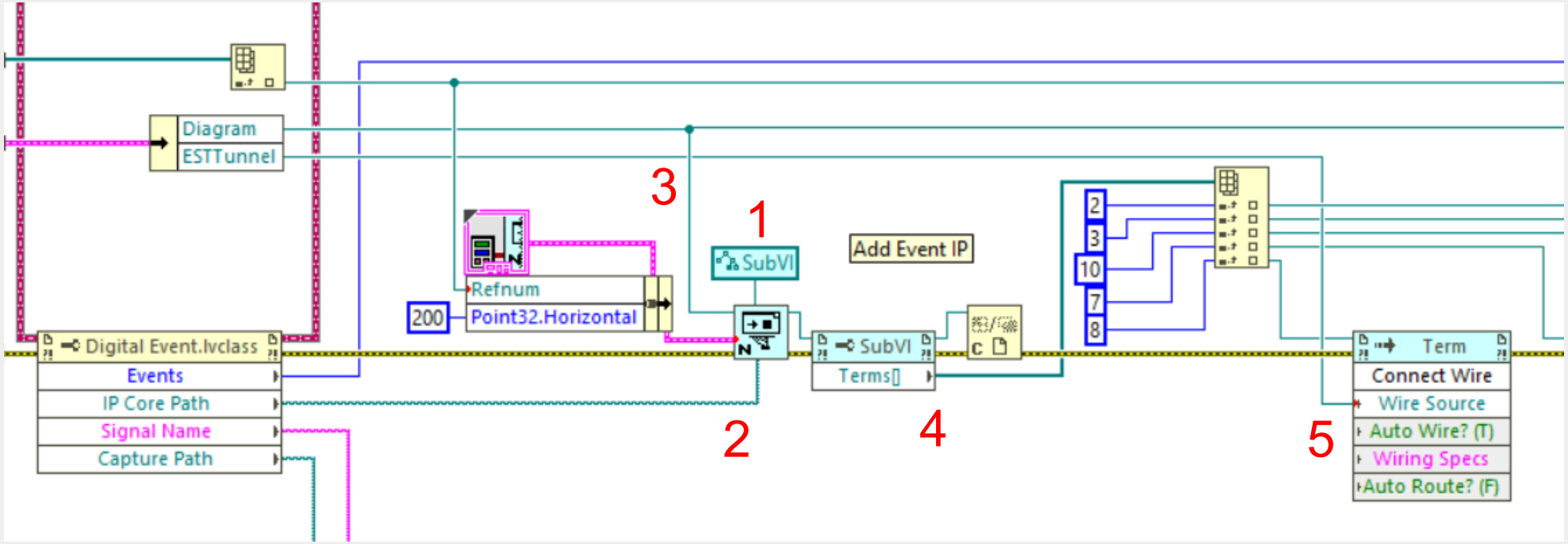
1. Add New VI Object
2. Set Object Class & Type (Usually match)
3. Get Terminals
4. Wire



Adding Items

SubVIs

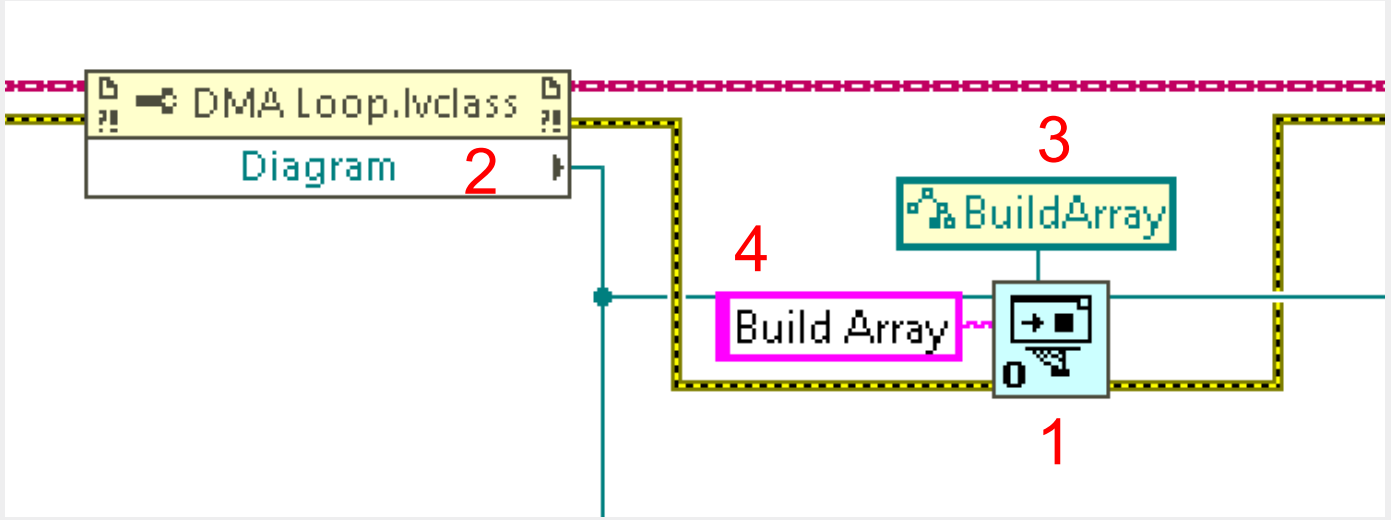
1. Add New VI Object
2. Pass Path
3. Pass owning diagram
4. Get Terminals
5. Wire



Finding Items

Existing Items

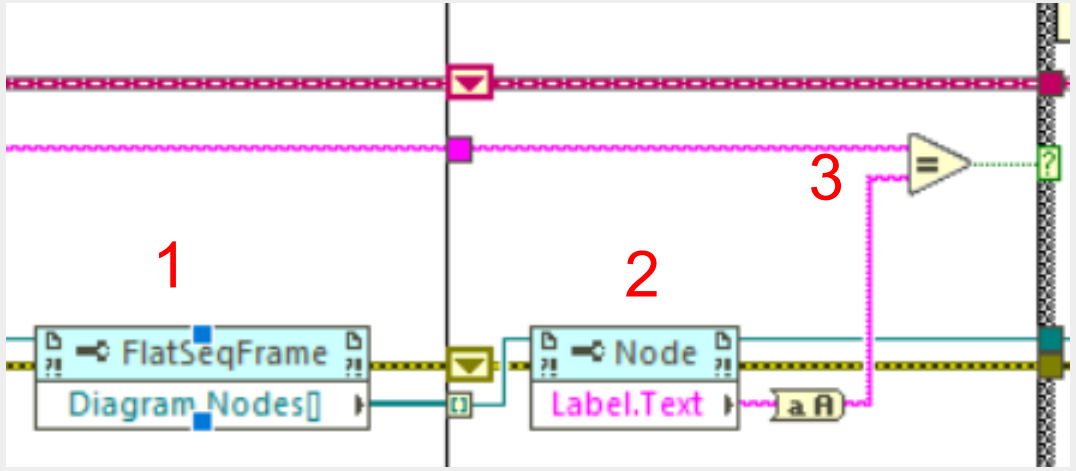
1. Use the Open Function
2. Use hierarchy concepts to narrow search context
3. Select type of item
4. Enter name of item



Finding Items

Existing Items

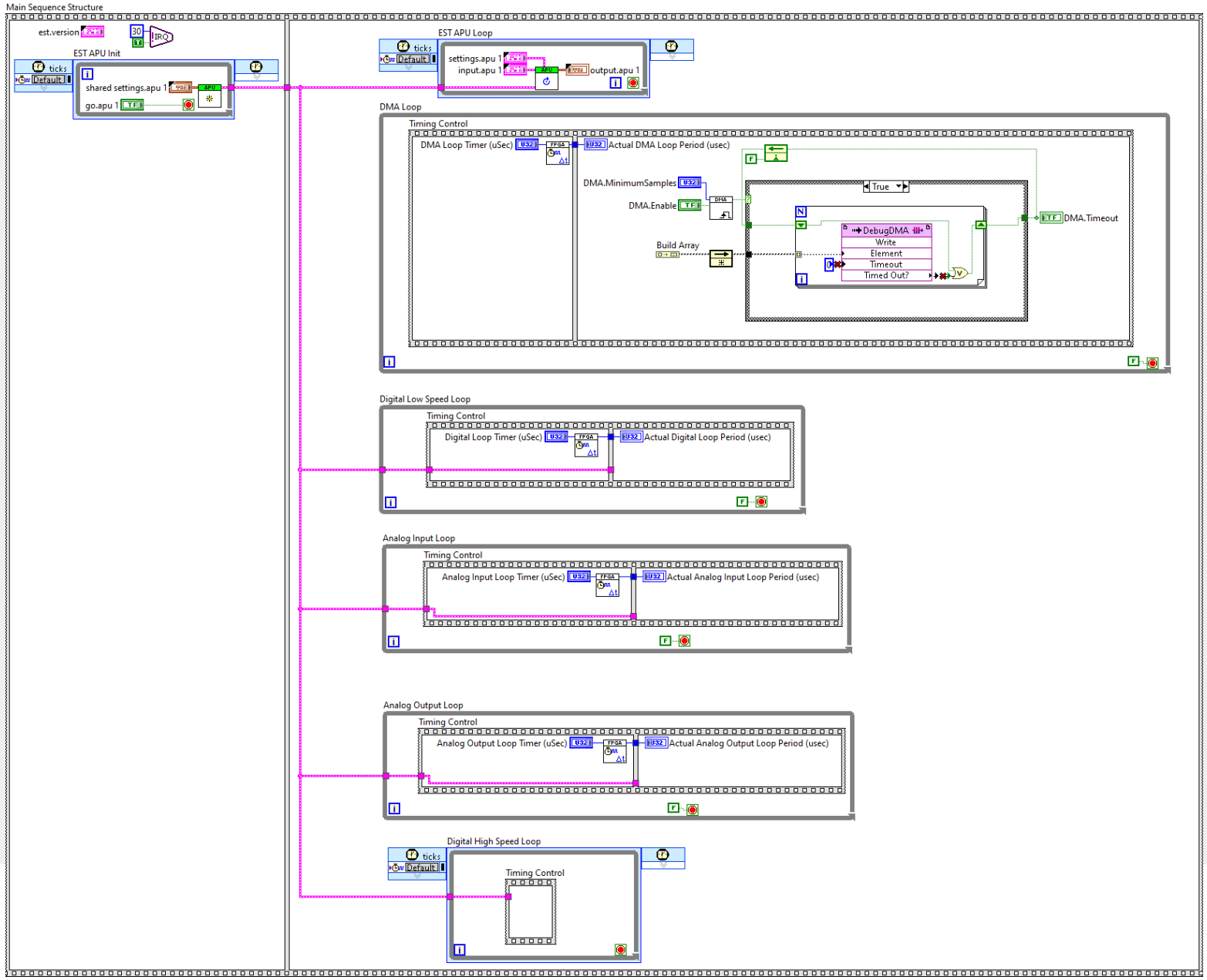
1. Grab all the objects on for a diagram reference
2. Iterate through objects list and get label text
3. Compare against expected



Using Templates

Template Design

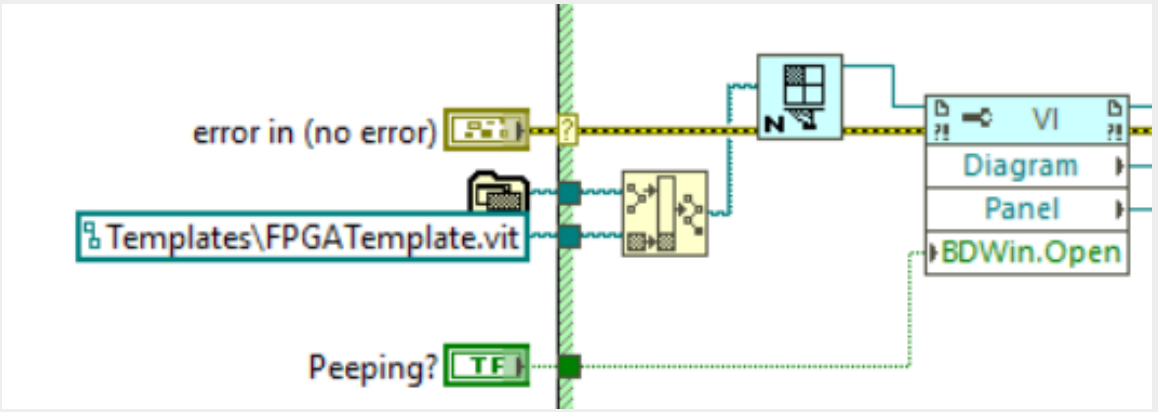
1. Name Everything (Uniquely) for search
2. Design core hierarchy
3. Design structures / looping
4. Include code that won't change
5. Stub functions that require input
6. Pass wires to main structures
7. Add code that may need to be removed later

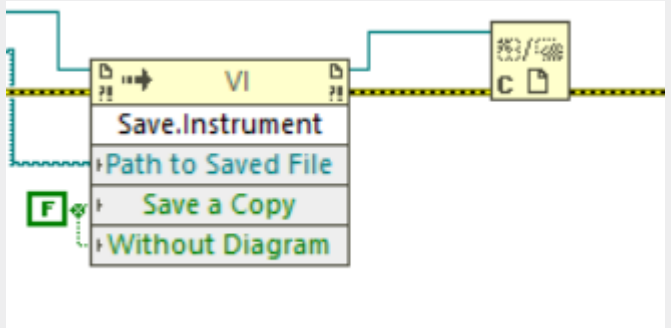


Using Templates

Using a Template

1. Open a new VI w/ template
2. Get Block Diagram / Front Panel Refs
3. Do Scripting Things
4. Save VI as *.vi (not *.vit)

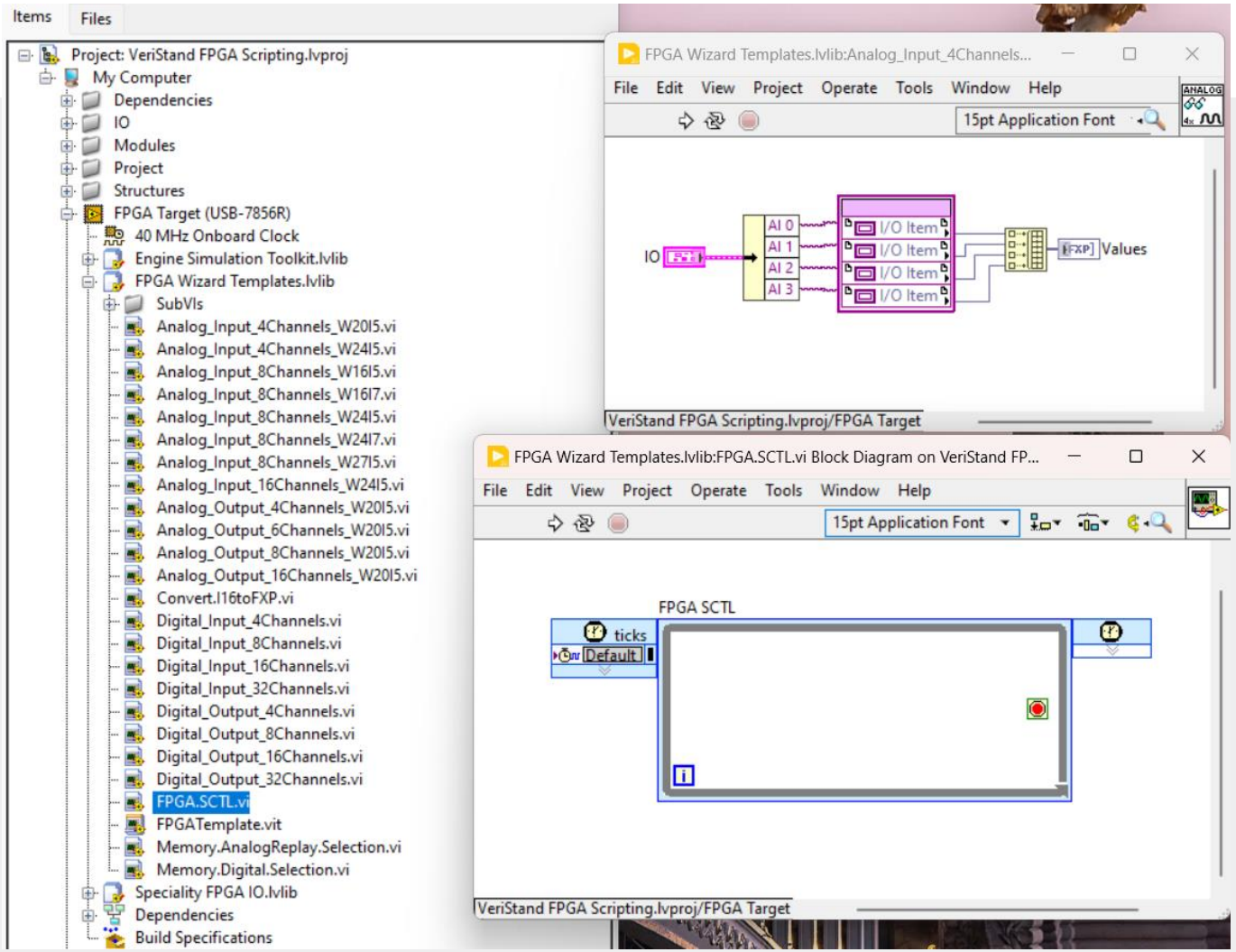




Using Templates

Working around XNodes

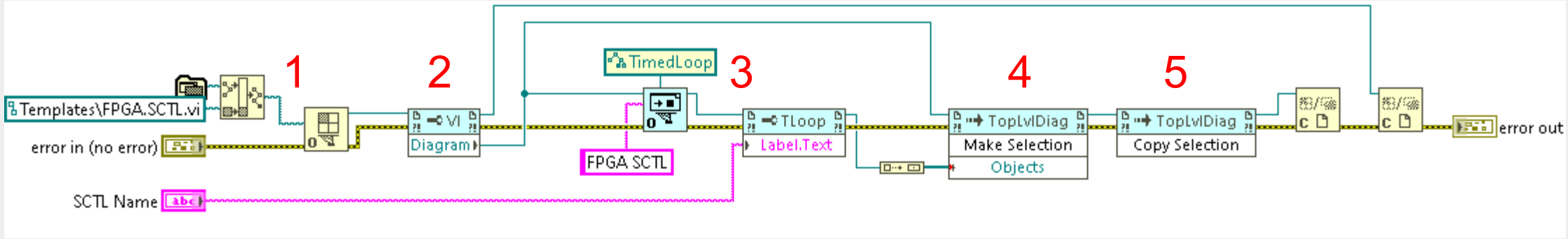
1. Create a template VI w/ Xnodes
2. Open VI with VI Server
3. Select block diagram objects
4. Copy objects
5. Paste them into the main diagram



Using Templates

Copying Contents

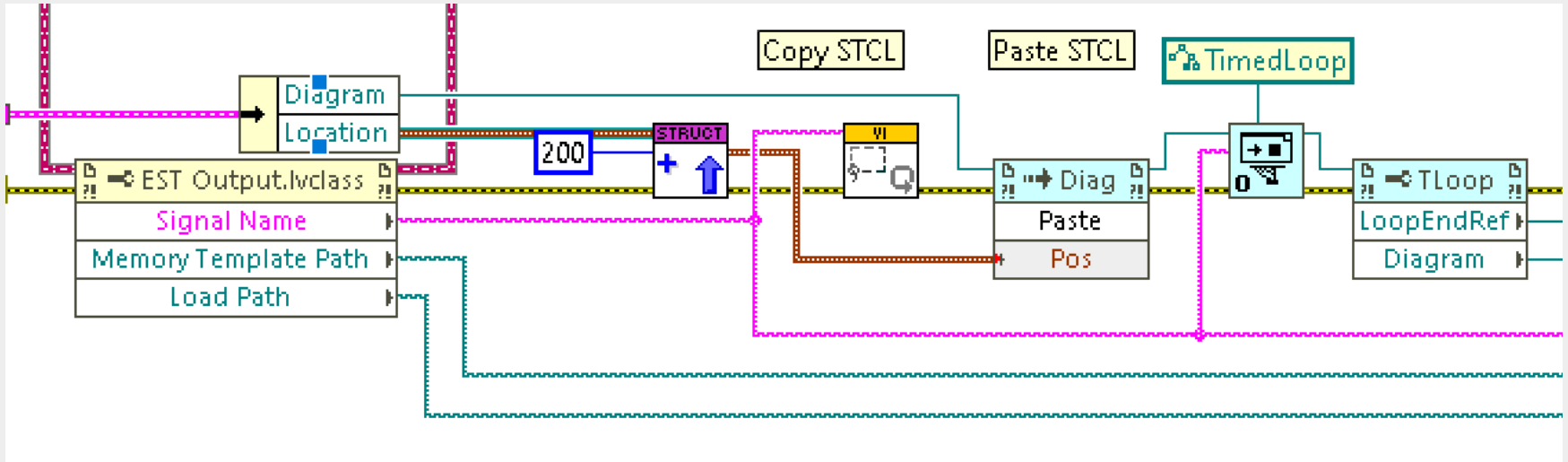
1. Open the Template
2. Get Diagram Reference
3. Navigate hierarchy
4. Select Items of interest
5. Copy



Using Templates

Pasting Contents (After Copy)

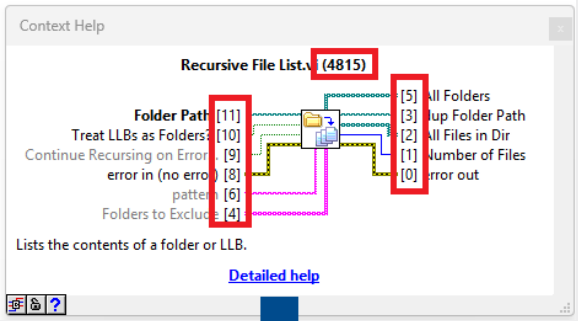
1. Navigate hierarchy to desired paste location reference
2. Paste at position



Wiring Functions

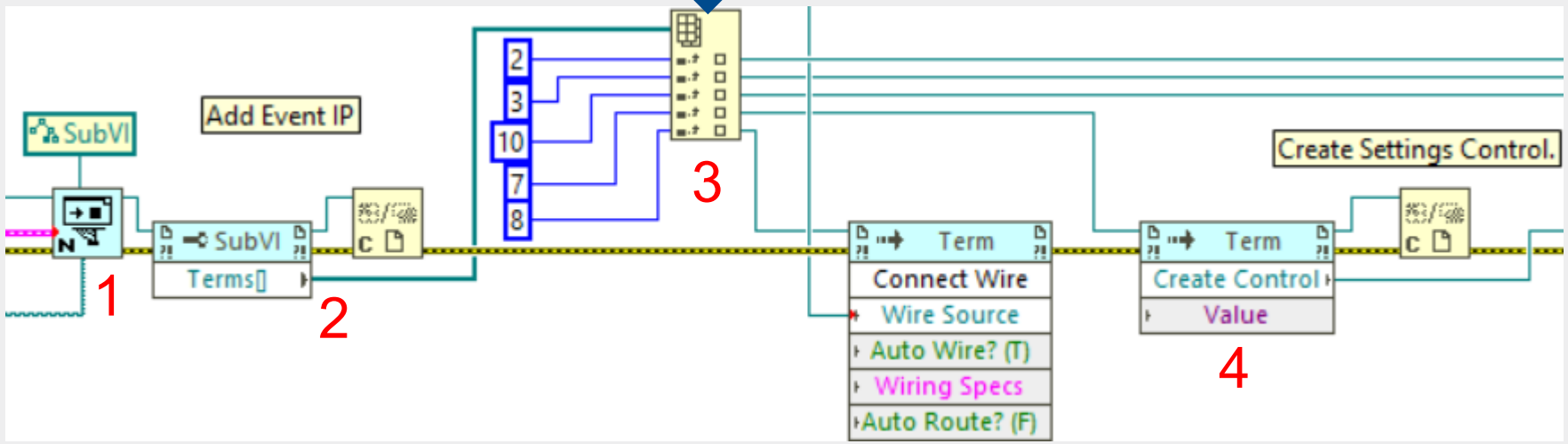
Creating Controls / Indicators

1. Place a template or SubVI first
2. Get the terminals
3. Find the desired terminal from terminals array
4. Create a Control or Indicator from terminal



Will automatically:

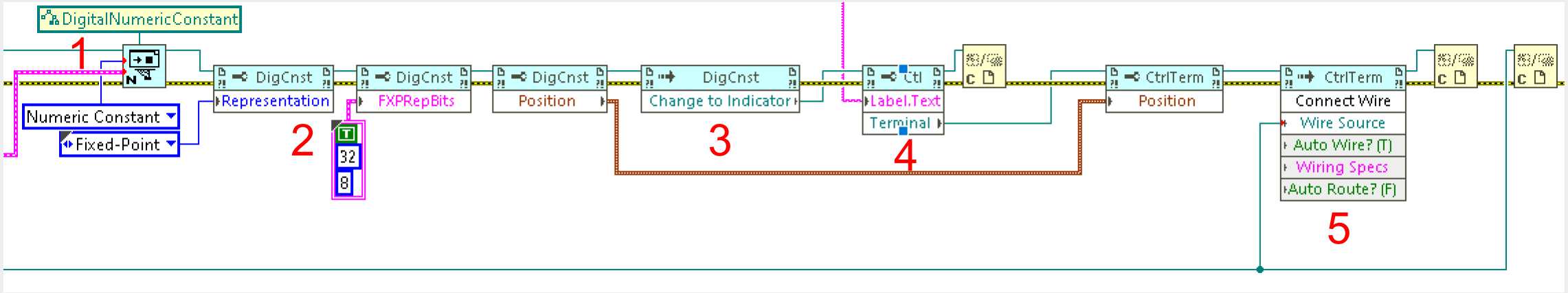
- Set correct data type
- Wire



Wiring Functions

Creating Constants (or controls/Indicators the hard way)

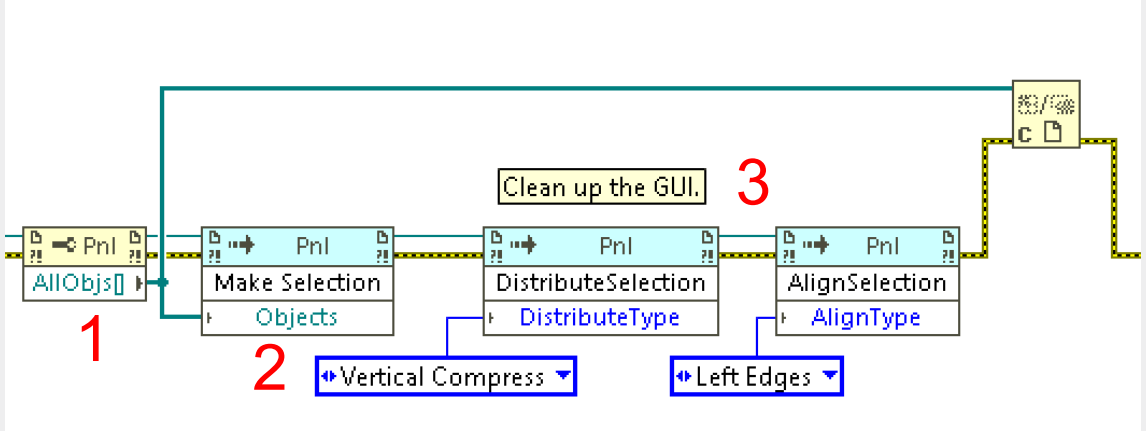
1. Add constant to referenced diagram
2. Set specific type
3. Invoke Change To Indicator/Control
4. Get Terminal
5. Connect Wire



Cleaning up Code

Cleaning up the GUI

1. Get a reference to all objects on the Panel
2. Select objects
3. Distribute as desired



Cleaning up Code

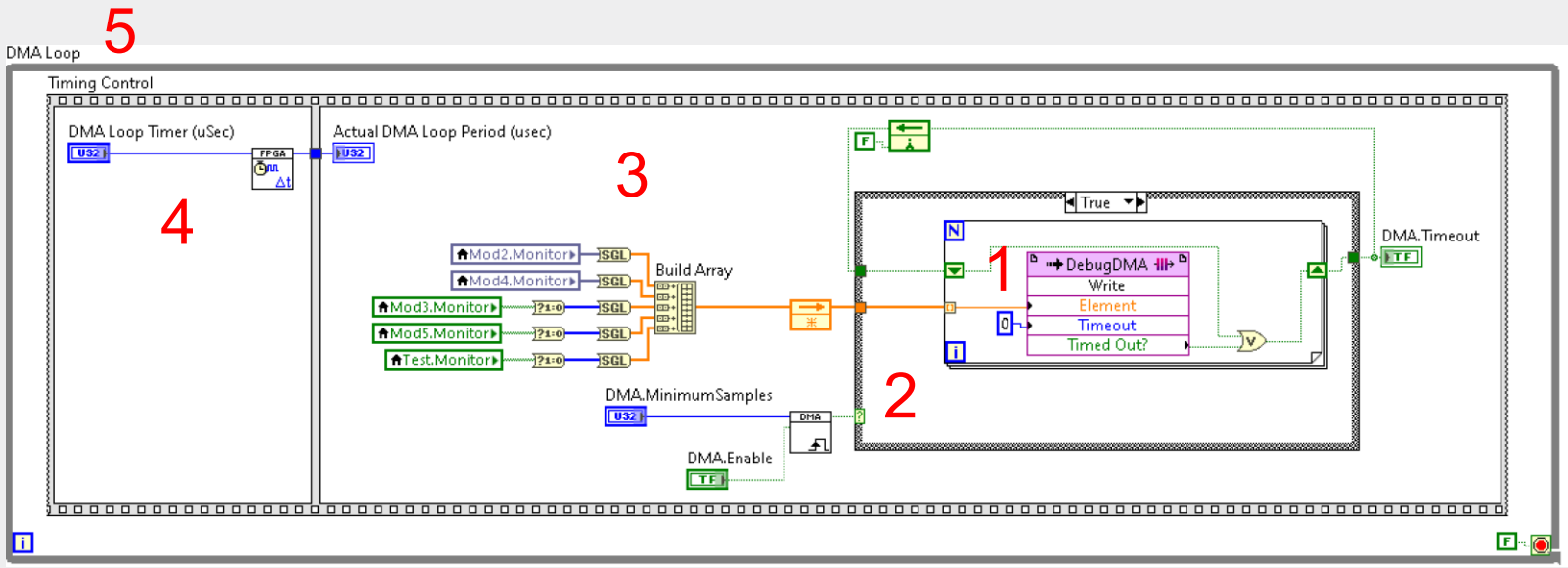
Cleaning up the Block Diagram

Concept:

- Start at the deepest, smallest part of the hierarchy
- Work progressively outward

Example:

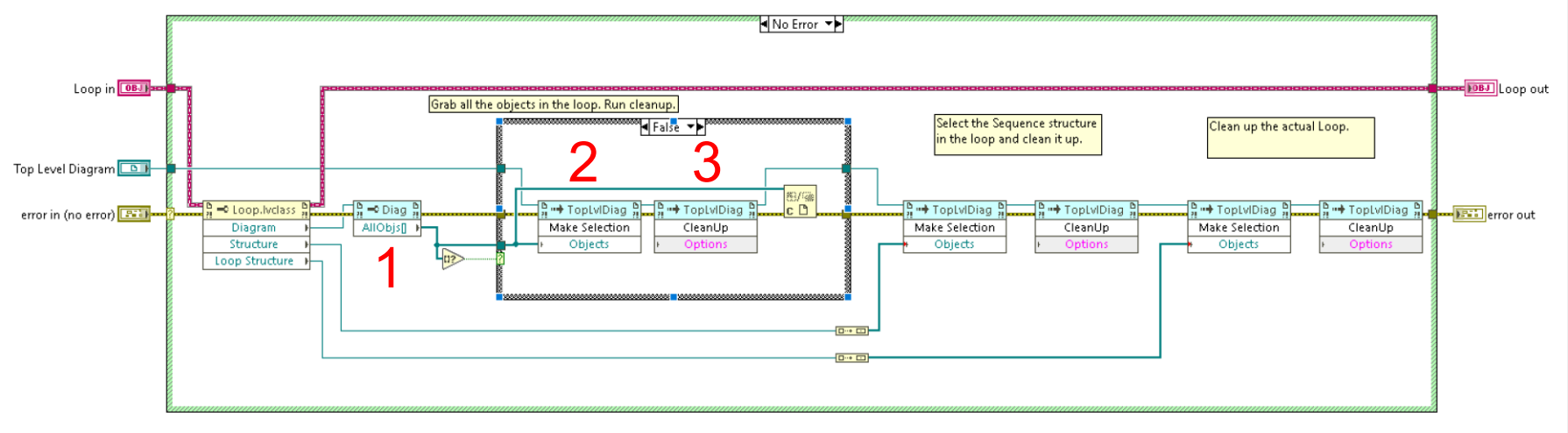
1. Clean For Loop
2. Clean Case
3. Clean Sequence Diagram 2
4. Clean Sequence Diagram 1
5. Clean While Loop



Cleaning up Code

Cleaning up the Block Diagram

1. Get a reference to all objects on the referenced diagram
2. Select objects
3. CleanUp

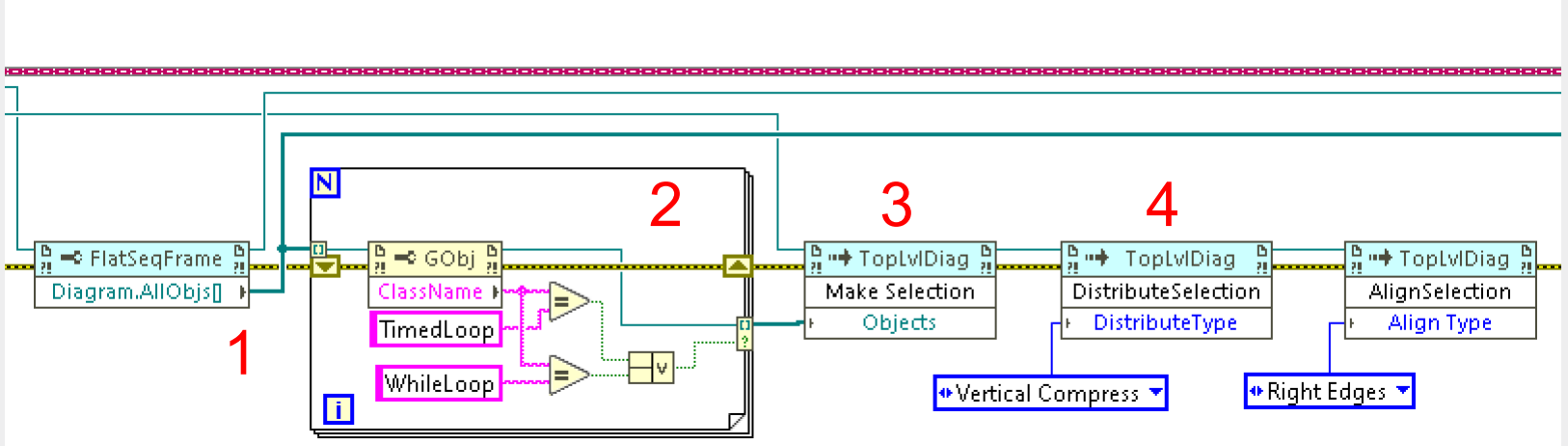


Cleaning up Code

Cleaning up the Block Diagram

Once most things are cleaned up in the core structures, distribute

1. Get a reference to all objects on the referenced diagram
2. Search for the core structures (e.g. while loops)
3. Select objects
4. Distribute



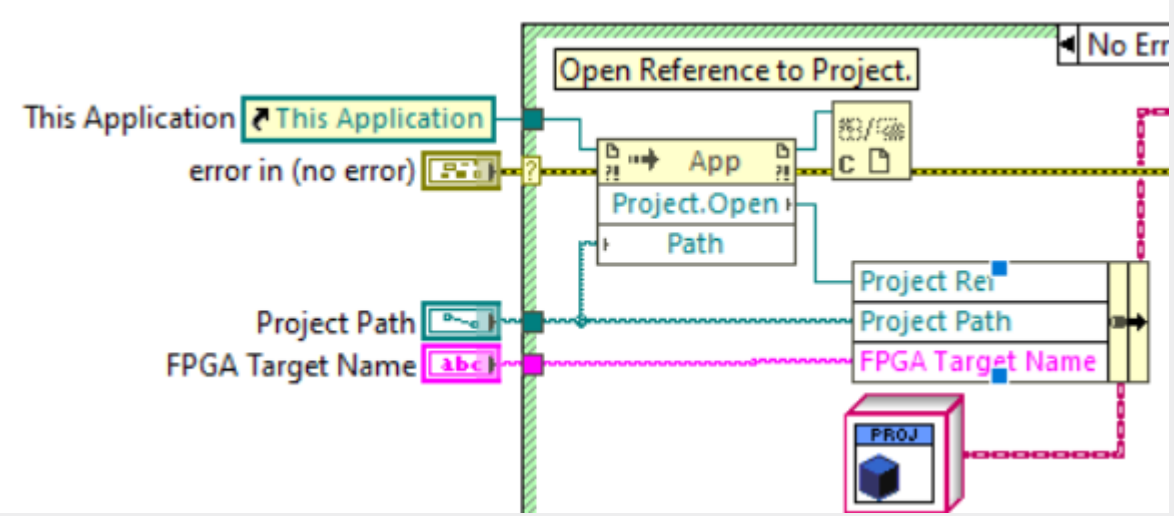
LabVIEW Project Modification

Opening Project Reference

Use the App Invoke Node

Select Project.Open

Give it your LV Project Path



Discovering Project Items

Iterate through Targets

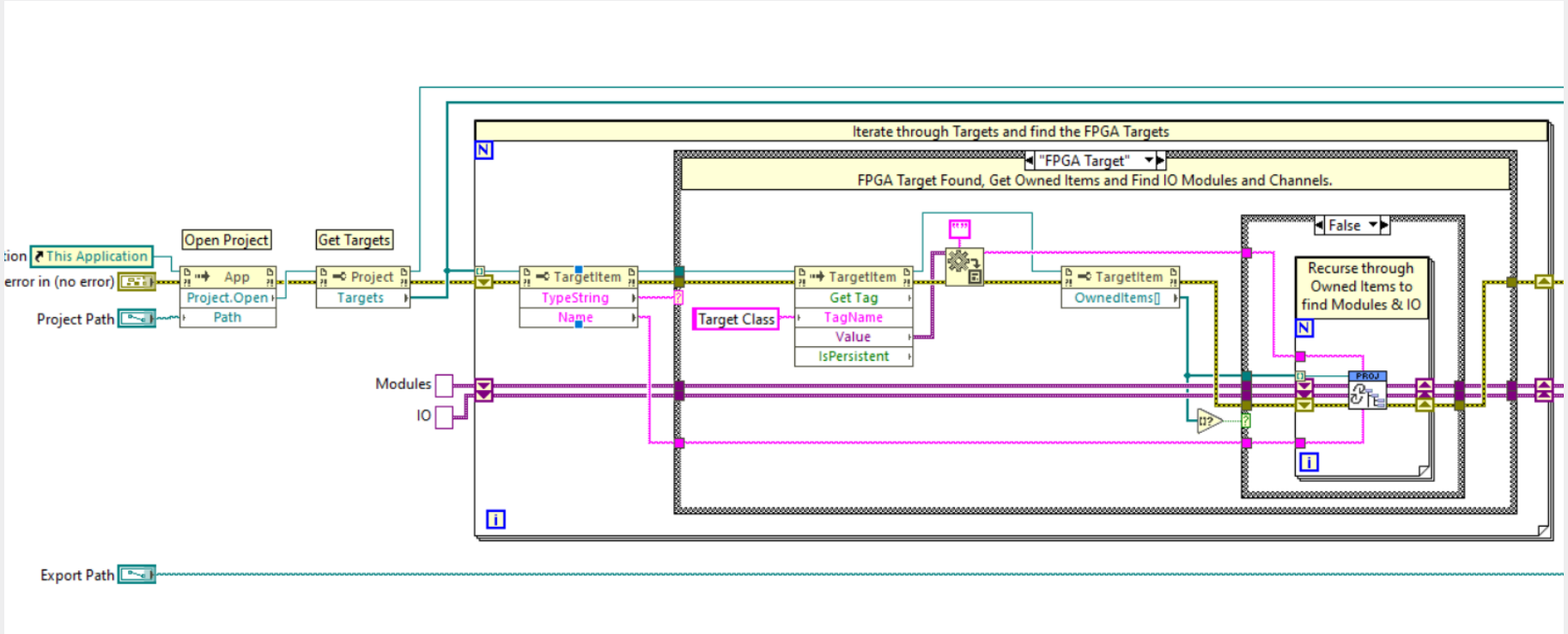
Iterate through Owned Items

Recurse through to find

- Elemental IO
- RIO C Series Module

Read Tags to get Module information

Build table of cached items

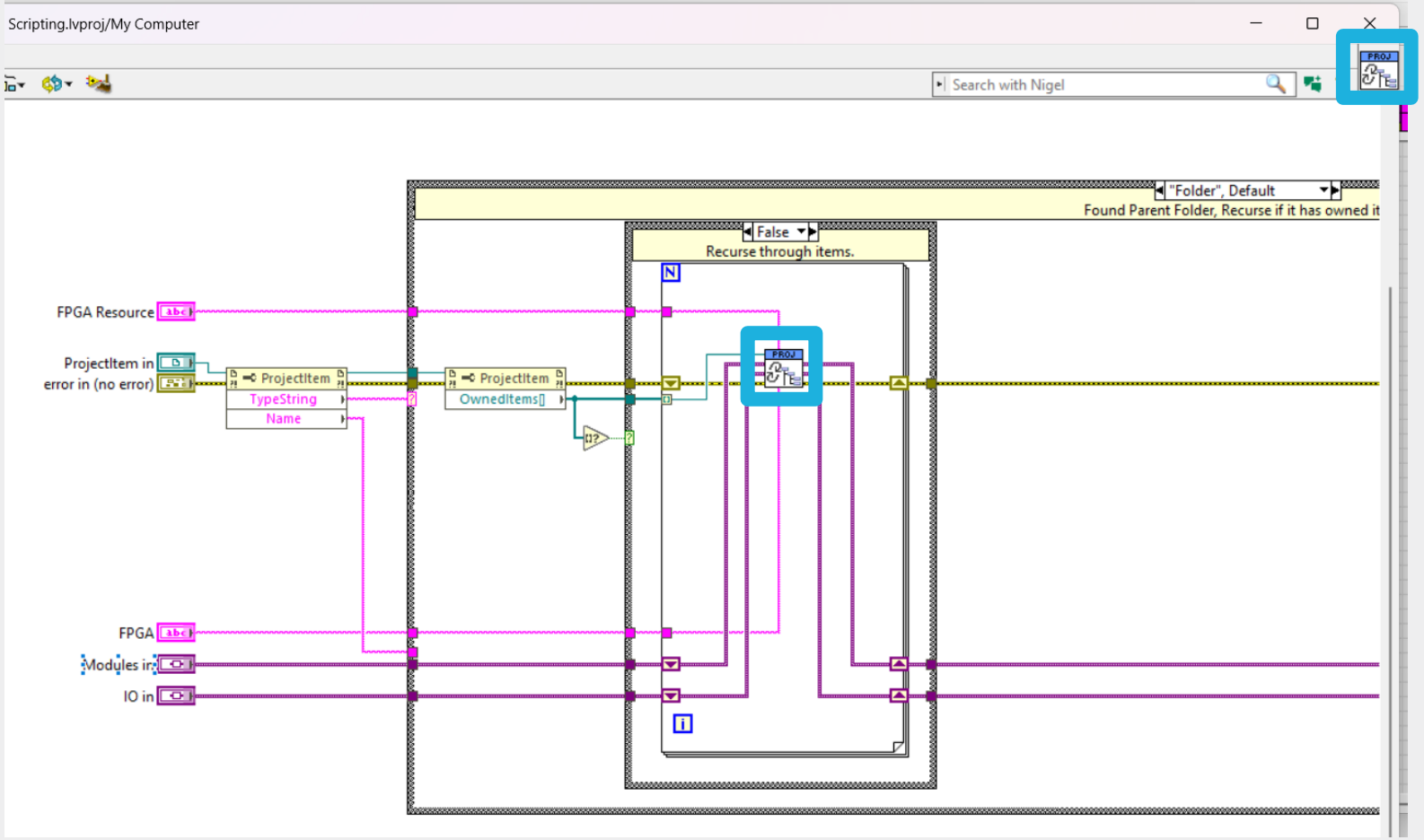


Discovering Project Items

Recurse Example Included!!

Source Code @

https://github.com/NISystemsEngineering/ni-LunchAndLearn-ScriptLvFpgaFromCsvExamples



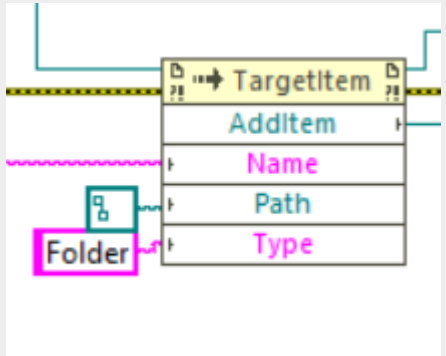
Adding Items to Project

Easy Way

Navigate to the container where you want to add things

Use Add Item Invoke

Decent documentation!!



But no FPPA Items…

- DMA
- Memory
- Etc

Type	abc	Yes	Specifies the project item type. The following values are the most common **Type** values: • VI • Folder • Library • XControl • EXE • DLL • Source Distribution • Installer • Zip File • Hyperlink

Adding Items to Project

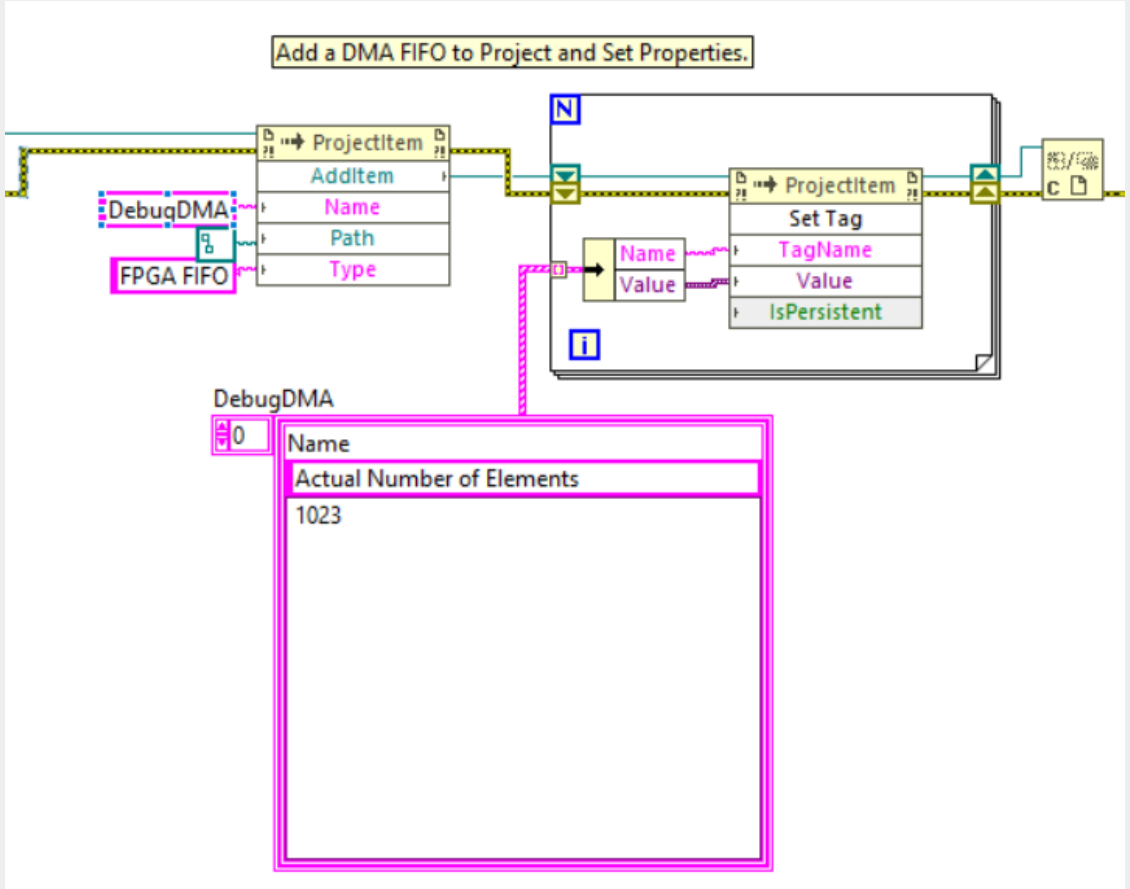
FPGA Way

Navigate to the container where you want to add things

Use Add Item Invoke

Add <Magic> Tags

No documentation!!



Adding Items to Project

For Tags…Read the Source

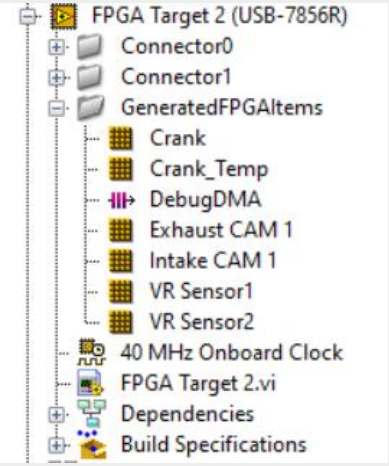
Open a new LabVIEW Project

Add your items

Save

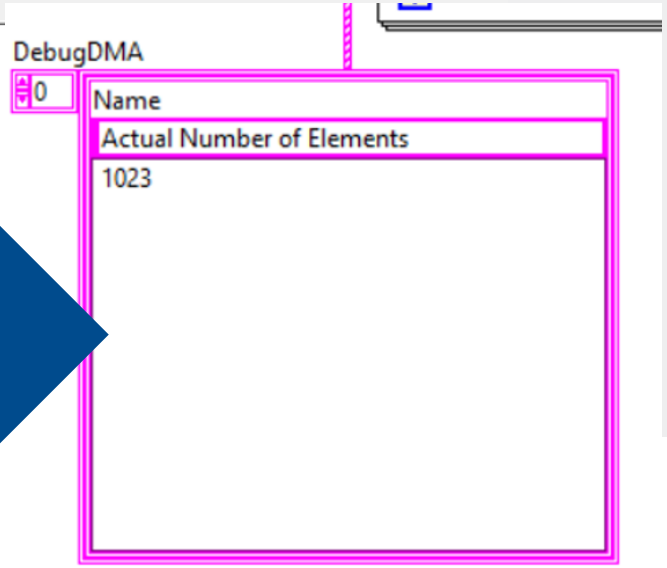
Open the .lvproj in a text editor

Copy, Modify, and Paste



R and C Series.lvproj

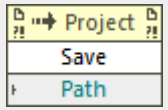
```
1606            <Property Name="Type Descriptor" Type="Str">1000800000000001000940060003553136000100000000000000000</Property>
1607        </Item>
1608        <Item Name="DebugDMA" Type="FPGA FIFO">
1609            <Property Name="Actual Number of Elements" Type="UInt">1023</Property>
1610            <Property Name="Arbitration for Read" Type="UInt">1</Property>
1611            <Property Name="Arbitration for Write" Type="UInt">1</Property>
1612            <Property Name="Control Logic" Type="UInt">0</Property>
1613            <Property Name="Data Type" Type="UInt">11</Property>
1614            <Property Name="Disable on Overflow/Underflow" Type="Bool">false</Property>
1615            <Property Name="fifo.configuration" Type="Str">"ControlLogic=0;NumberOfElements=1023;Type=2;ReadArbs=Arbitrate if Mu
1616            <Property Name="fifo.configured" Type="Bool">true</Property>
1617            <Property Name="fifo.projectItemValid" Type="Bool">true</Property>
1618            <Property Name="fifo.valid" Type="Bool">true</Property>
1619            <Property Name="fifo.version" Type="Int">12</Property>
1620            <Property Name="FPGA.PersistentID" Type="Str">{43D2C6D9-9E30-4F2C-9E9A-27A6F06ED45E}</Property>
1621            <Property Name="Local" Type="Bool">false</Property>
1622            <Property Name="Memory Type" Type="UInt">2</Property>
1623            <Property Name="Number Of Elements Per Read" Type="UInt">1</Property>
1624            <Property Name="Number Of Elements Per Write" Type="UInt">1</Property>
1625            <Property Name="Requested Number of Elements" Type="UInt">1023</Property>
1626            <Property Name="Type" Type="UInt">2</Property>
1627            <Property Name="Type Descriptor" Type="Str">1000800000000001000940090003534 74C0001000000000000000000</Property>
1628        </Item>
```



Adding Items to Project

Add VI

Save Project



LabVIEW Classes

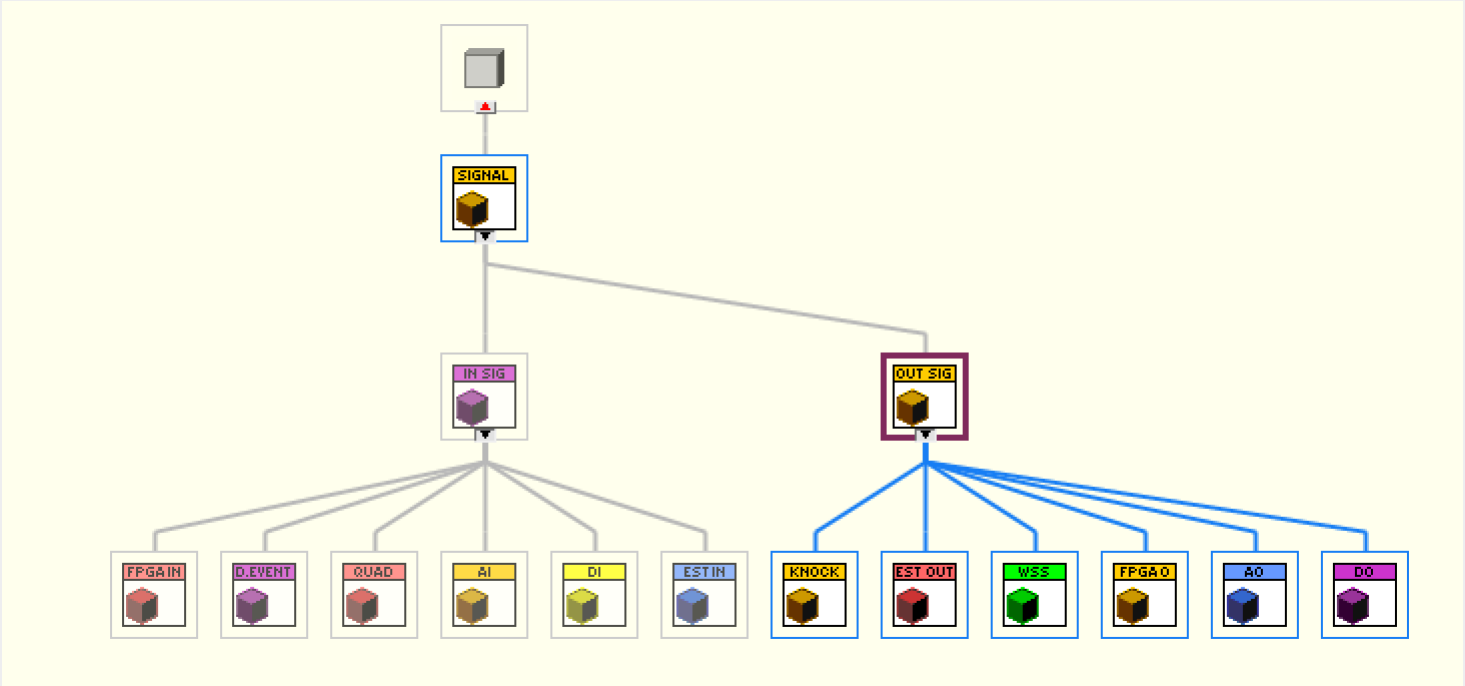
(Not covered today but…)

Logical LabVIEW Class Application

FPGA Signal IP uses Classes for Dynamic Dispatch

Adding more IP is just creating a new child class

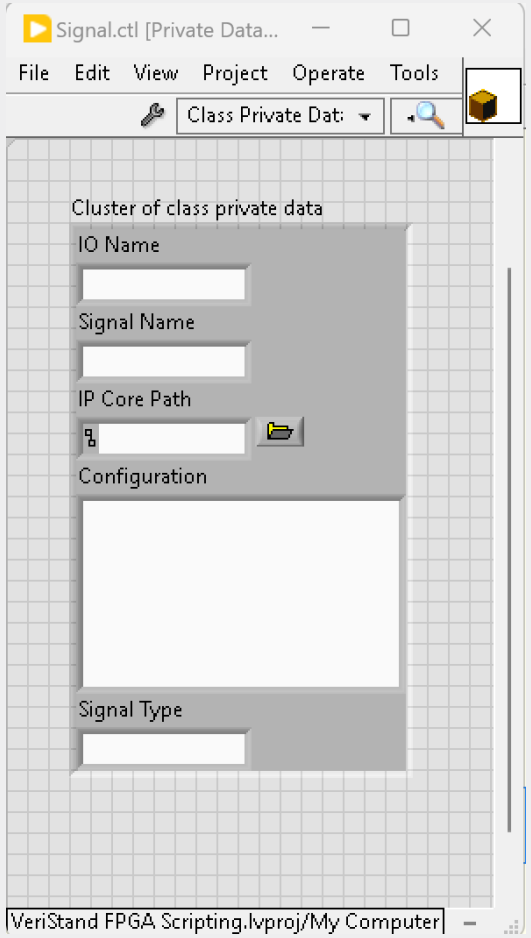
Requires little or no modification to rest of code



Signal IP Container Class

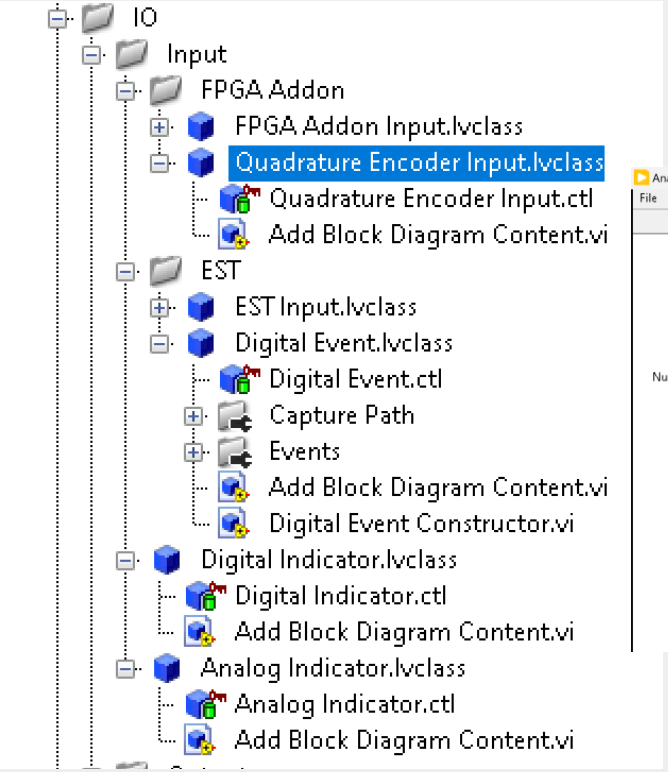
Signal IP container required settings:

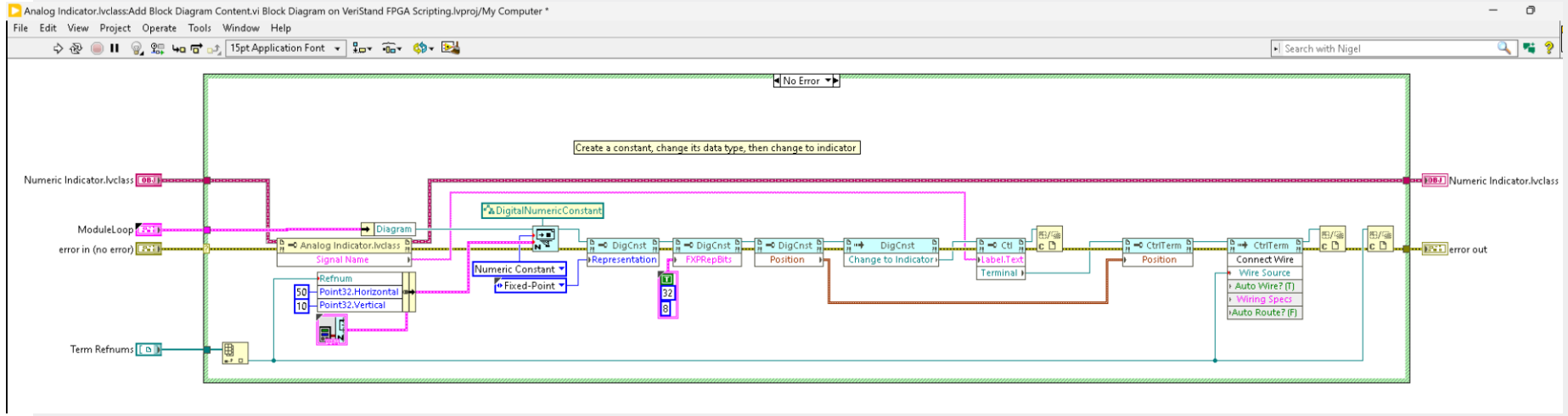
- IO name
- Signal name
- IP Core Path (Specific Signal IP Logic)
- Configuration JSON for Specific Signal IP Logic
- Signal type



Specific Signal IP Class

Main Override: “Add Block Diagram Content:



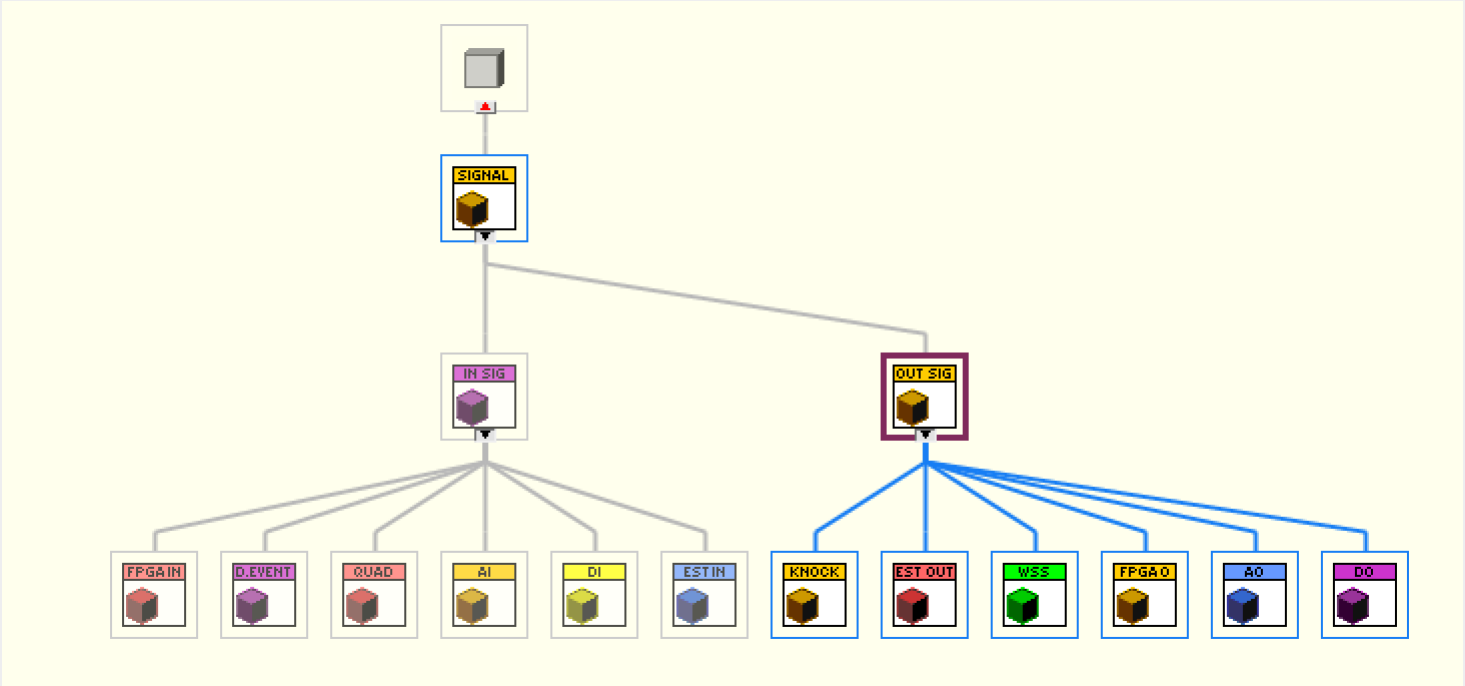


Logical LabVIEW Class Application

Example Included!!

Source Code @

https://github.com/NISystemsEngineering/ni-LunchAndLearn-ScriptLvFpgaFromCsvExamples



Review Code

Questions?

Daniel Eaton

Chief Field Applications Engineer

CLA, CLED, 20 years “doing” w/ NI

Daniel.eaton@emerson.com

We are here to help!

- Pull us in to design conversations
- Let us help de-risk projects
- Tell us what you want to learn about next